



INSTITUTO FEDERAL

Santa Catarina
Câmpus São José

MINISTÉRIO DA EDUCAÇÃO
SECRETARIA DE EDUCAÇÃO PROFISSIONAL E TECNOLÓGICA
INSTITUTO FEDERAL DE SANTA CATARINA
CAMPUS SÃO JOSÉ

ICD - INTRO. À CIÊNCIA DE DADOS E APRENDIZADO DE MÁQUINA

PARTE 3: Machine Learning Supervisionado

Prof. Ramon Mayor Martins, Dr.

[ramon.mayor at: ifsc.edu.br](mailto:ramon.mayor@ifsc.edu.br)

Apresentação
Disponível em:

Processo de Desenvolvimento de Modelo de Machine Learning



Agenda da Parte 2

- ❑ Introdução ao Aprendizado Supervisionado
- ❑ Classificação: Conceitos e Fundamentos
 - ❑ Redes Neurais Artificiais
 - ❑ Demais Algoritmos Clássicos de Classificação
 - ❑ Métricas de Avaliação para Classificação



Etapas 2-3-4:Treinamento/Avaliação/Predição

Tipo de Aprendizado: Aprendizado Supervisionado

Tarefa: Classificação

Treinamento: Aprendizado Supervisionado

O aprendizado supervisionado utiliza dados rotulados (X, y) para treinar modelos que podem fazer previsões em novos dados.

Dados de entrada: Features (X) + Labels (y)

Objetivo: Aprender $X + y$

Avaliação: Comparar previsões com valores reais

Tipos: Classificação e Regressão

Aplicações: Diagnóstico médico, detecção de fraude, previsão de preços

Treinamento: Classificação vs Regressão

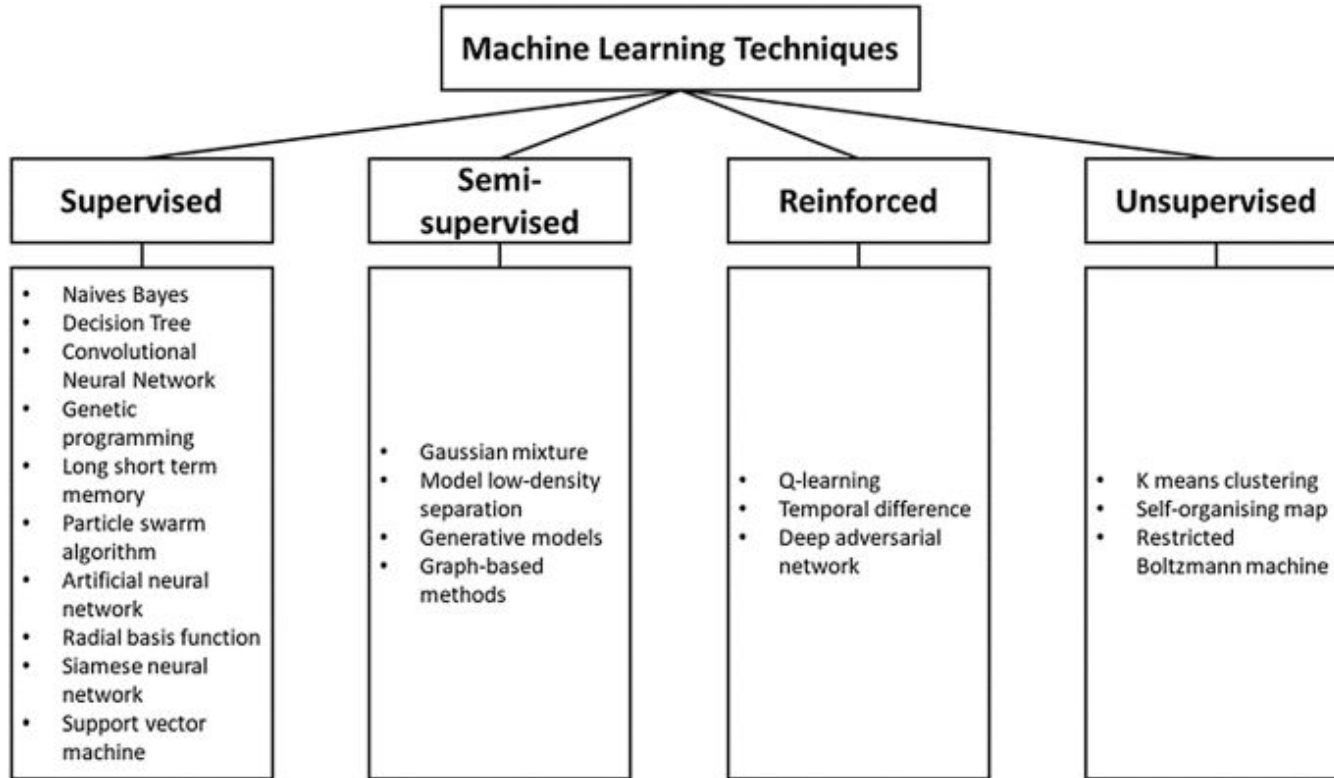
Classificação: Prediz categorias discretas (spam/não-spam, doença/saudável)

Regressão: Prediz valores contínuos (preço, temperatura, idade)

Target Categórico: Classificação

Target Numérico: Regressão

Treinamento: Técnicas para Classificação

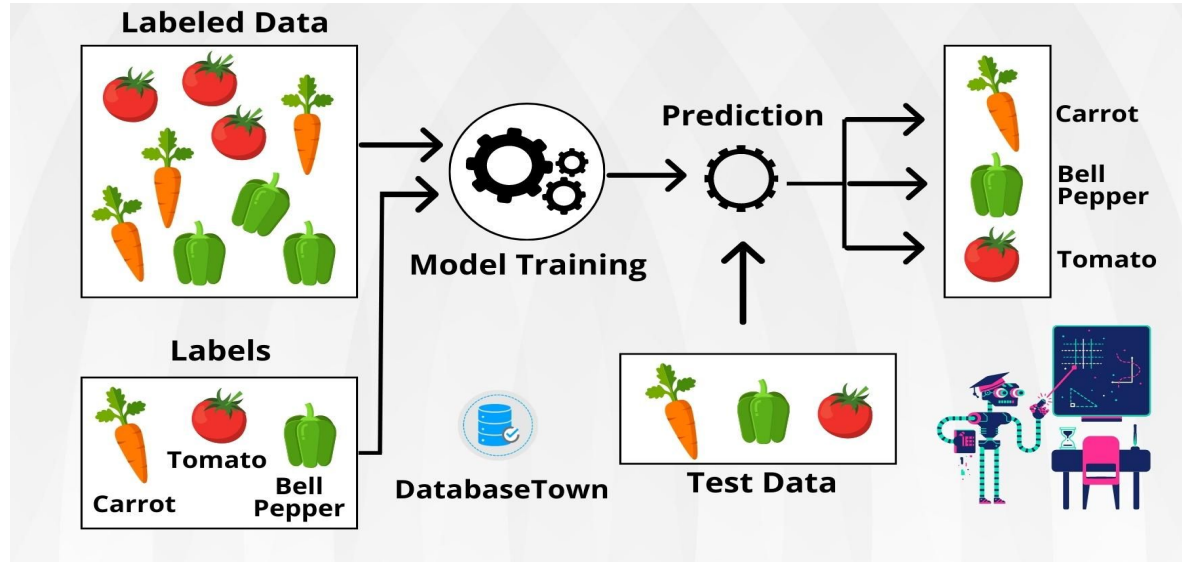


Treinamento: Pipeline de Classificação

1. Carregamento dos dados rotulados
2. Divisão em treino/validação/teste
3. Treinamento do modelo
4. Predição em dados novos
5. Avaliação do desempenho



Treinamento: Pipeline de Classificação

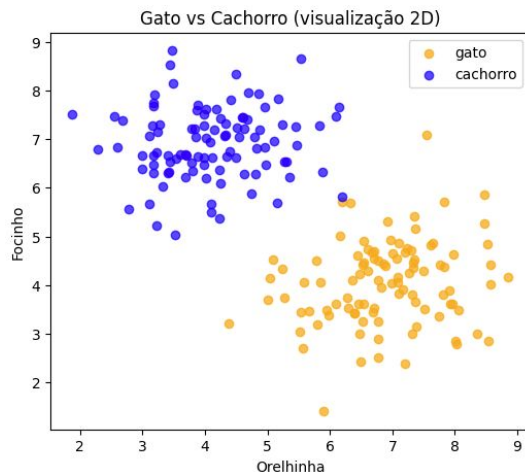


Treinamento: Pipeline de Classificação

Antes de ver as técnicas para classificação

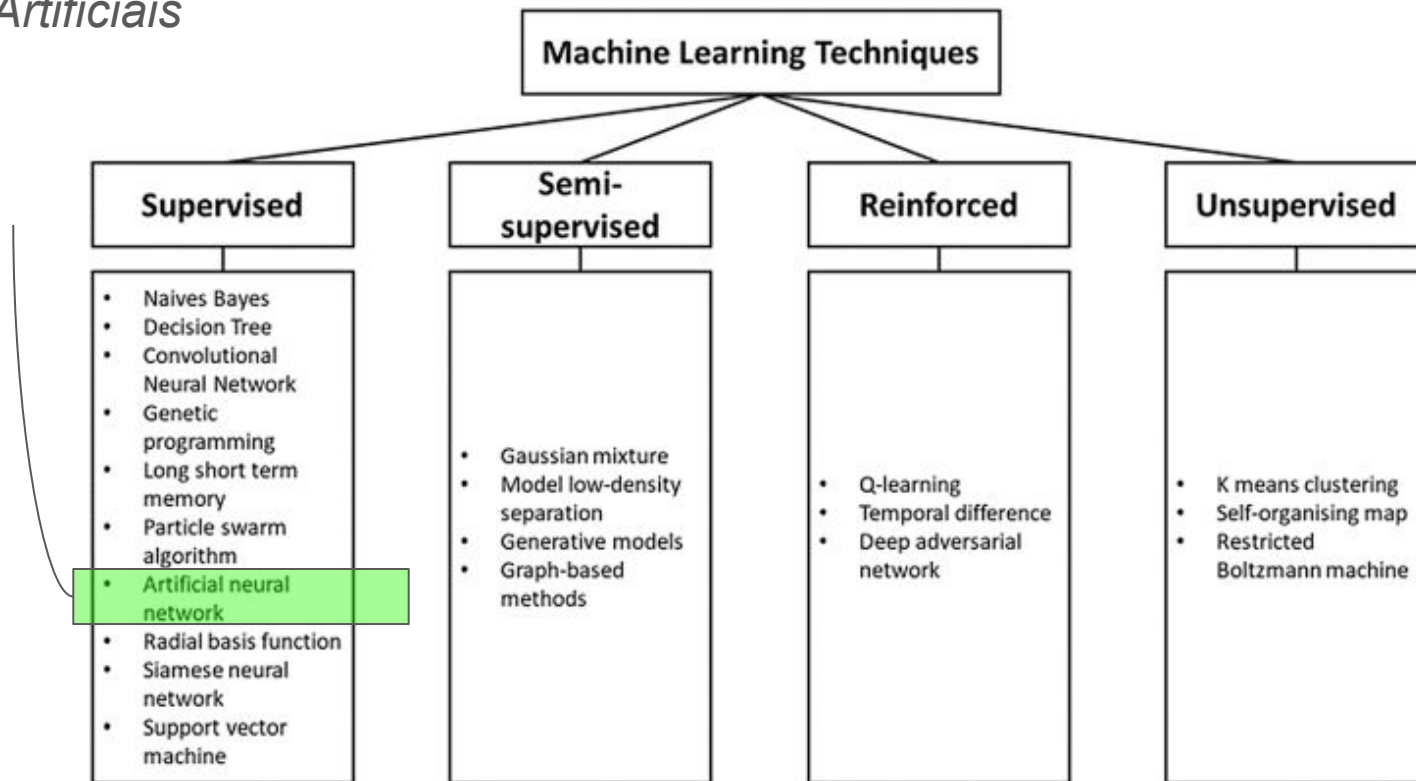
Lembremos de um dataset gato e cachorro.

Que tem suas características extraídas , dispostas em um plano cartesiano.



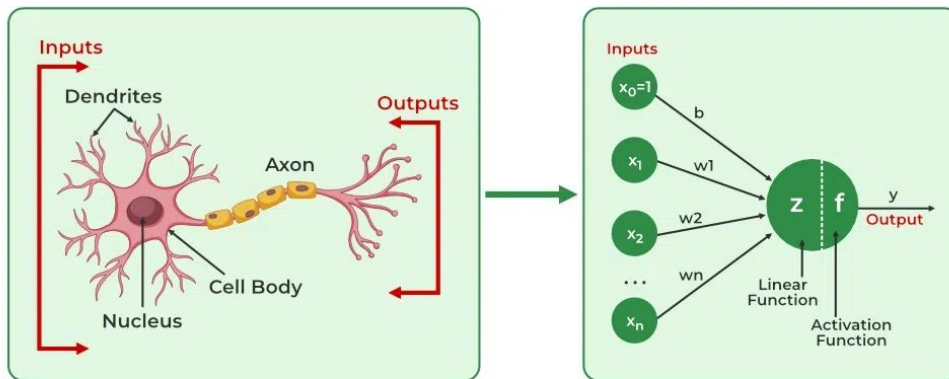
Treinamento: Técnicas para Classificação

*Vamos iniciar vendo as Redes
Neurais Artificiais*



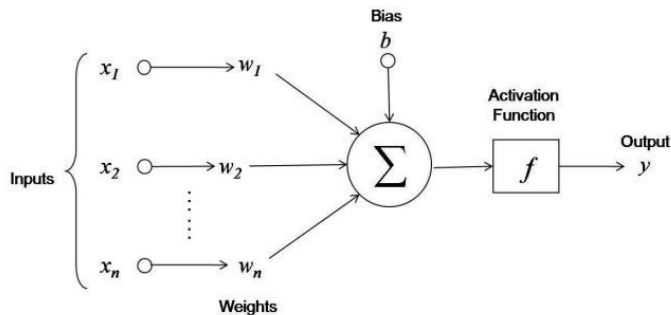
Treinamento: Redes Neurais Artificiais

As redes neurais artificiais são inspiradas no funcionamento dos neurônios biológicos e são compostas por camadas de neurônios interconectados.



Treinamento: Redes Neurais Artificiais

Componentes de uma Rede Neural



Neurônios (Nodes): Unidades de processamento

Pesos (Weights): Força das conexões

Bias: Termo de ajuste

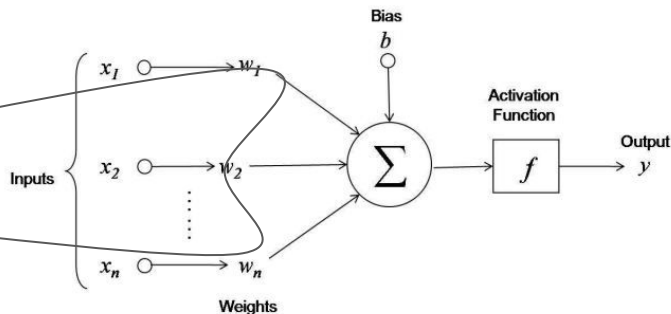
Função de Ativação: Habilita a passagem do sinal

Camadas: Input, Hidden, Output

Treinamento: Redes Neurais Artificiais

Componentes de uma Rede Neural

w Funciona como um botão de volume da entrada para o neurônio



Neurônios (Nodes): Unidades de processamento

Pesos (Weights): Força das conexões

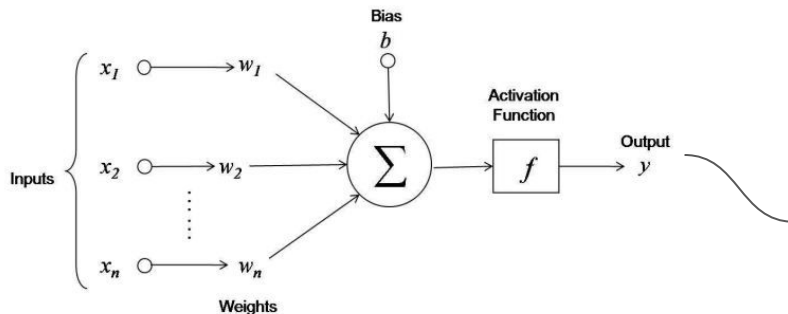
Bias: Termo de ajuste

Função de Ativação: Habilita a passagem do sinal

Camadas: Input, Hidden, Output

Treinamento: Redes Neurais Artificiais

Componentes de uma Rede Neural



Neurônios (Nodes): Unidades de processamento

Pesos (Weights): Força das conexões

Bias: Termo de ajuste

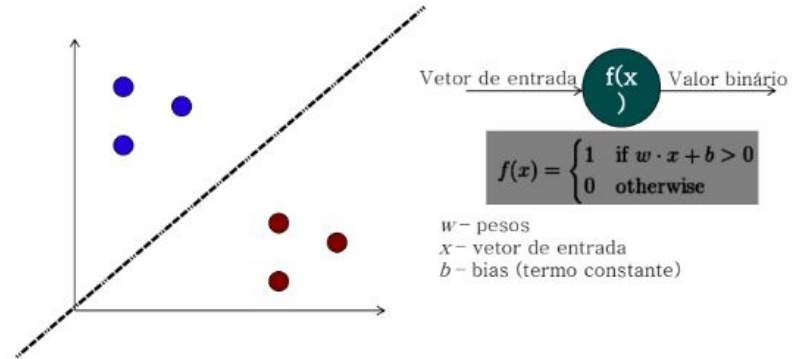
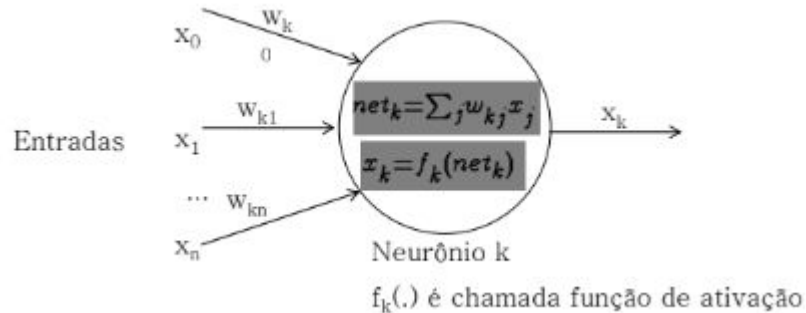
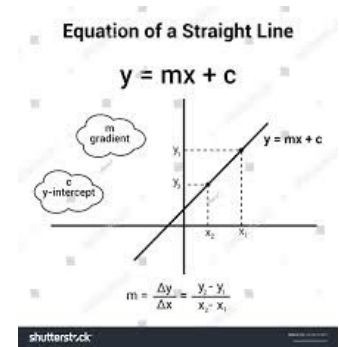
Função de Ativação: Habilita a passagem do sinal

Camadas: Input, Hidden, Output

Treinamento: Redes Neurais Artificiais

Funcionamento de uma Rede Neural

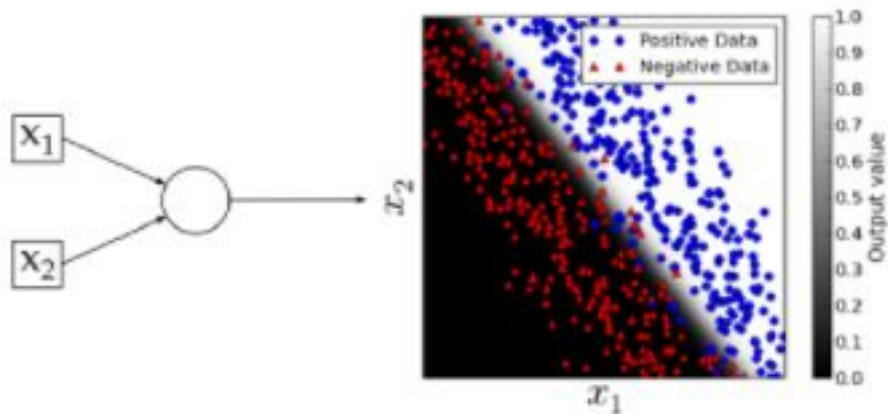
Cada neurônio desenvolve um hiperplano pela equação da reta



Treinamento: Redes Neurais Artificiais

Funcionamento de uma Rede Neural

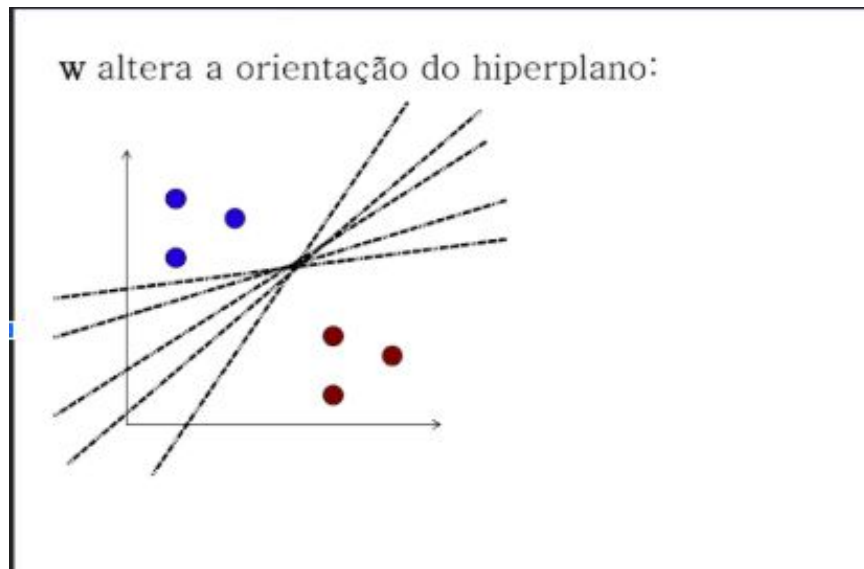
Com o hiperplano, é possível traçar uma separabilidade



Treinamento: Redes Neurais Artificiais

Funcionamento de uma Rede Neural

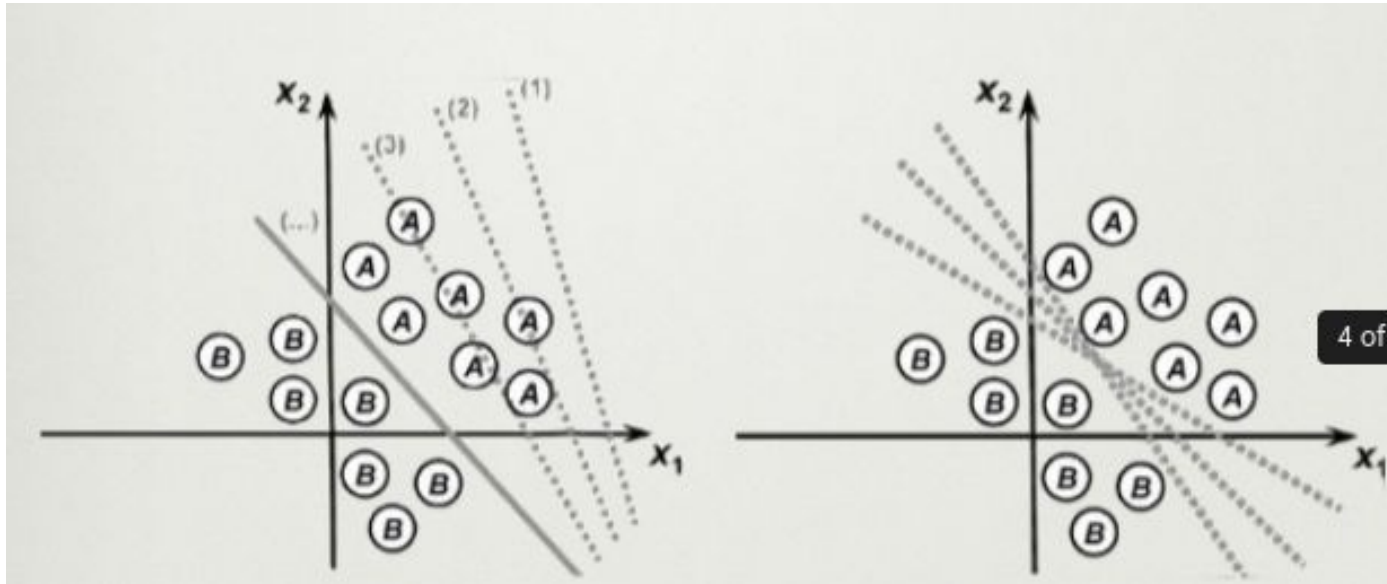
Alterando os pesos w dos neurônios com as entradas, vou inclinando/rotacionando os hiperplanos de modo q vai melhorando a separação ou não



Treinamento: Redes Neurais Artificiais

Funcionamento de uma Rede Neural

Efeito dos pesos (é a equação da reta, altera um valor, vc altera o cartesiano dele)



Treinamento: Redes Neurais Artificiais

Funcionamento de uma Rede Neural

Em uma Rede Neural, quanto mais treinamento tiver, melhor esses pesos vão se ajustando até formar uma boa separabilidade

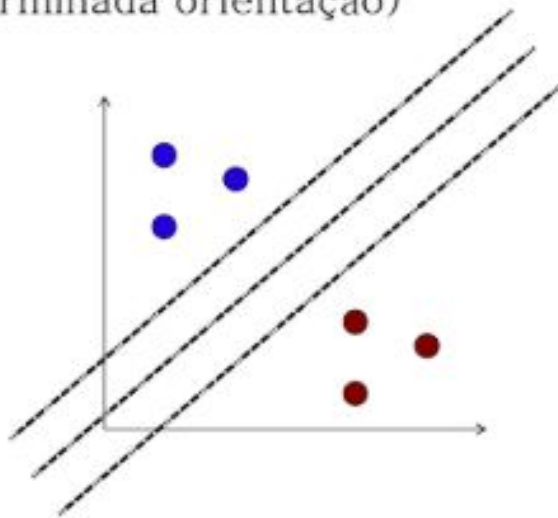


Treinamento: Redes Neurais Artificiais

Funcionamento de uma Rede Neural

O b (bias) desloca a hiperplano

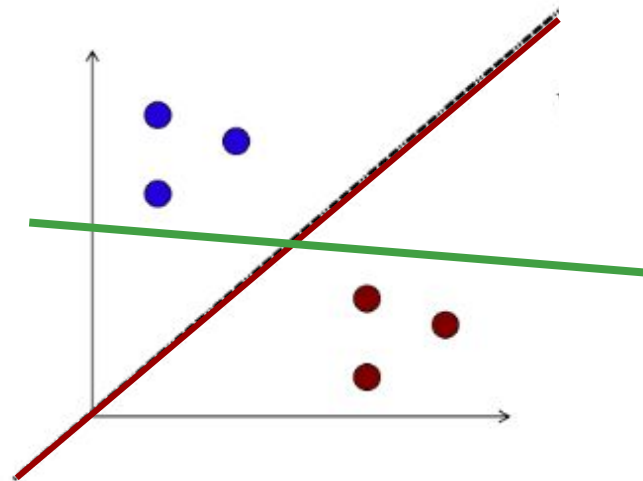
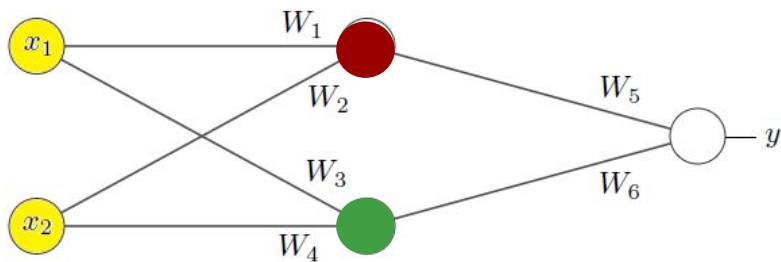
b altera a posição (em relação à origem, para uma determinada orientação)



Treinamento: Redes Neurais Artificiais

Funcionamento de uma Rede Neural

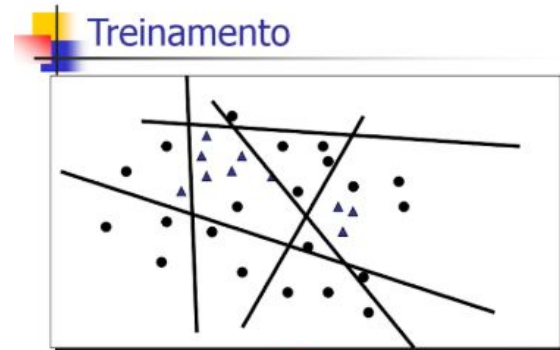
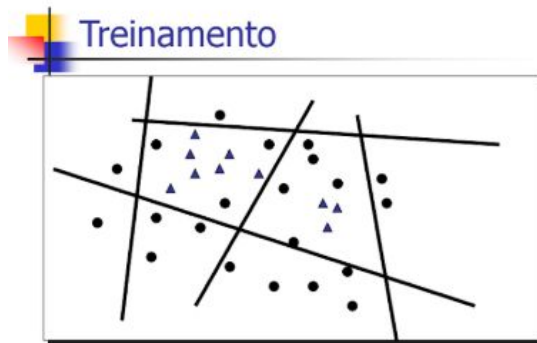
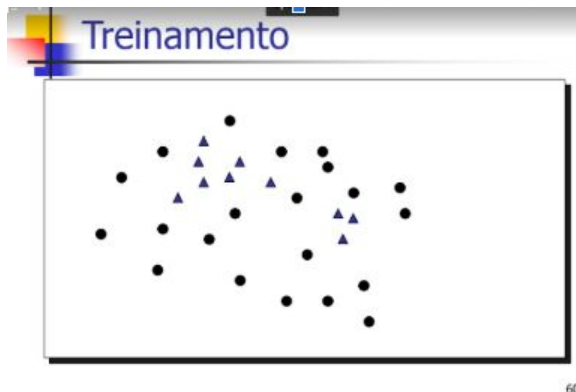
Quanto mais neurônios tiver, mais hiperplanos vai ter



Treinamento: Redes Neurais Artificiais

Funcionamento de uma Rede Neural

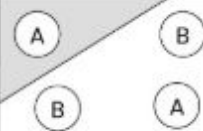
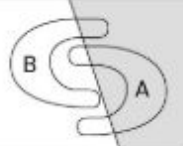
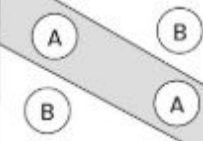
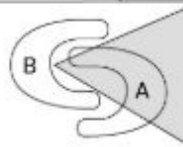
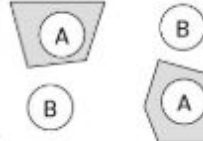
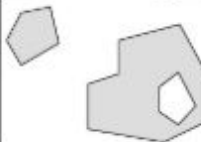
Até que muitos neurônios, com muitos hiperplanos, aumentam a separabilidade, aumentam a decisão



Treinamento: Redes Neurais Artificiais

Funcionamento de uma Rede Neural

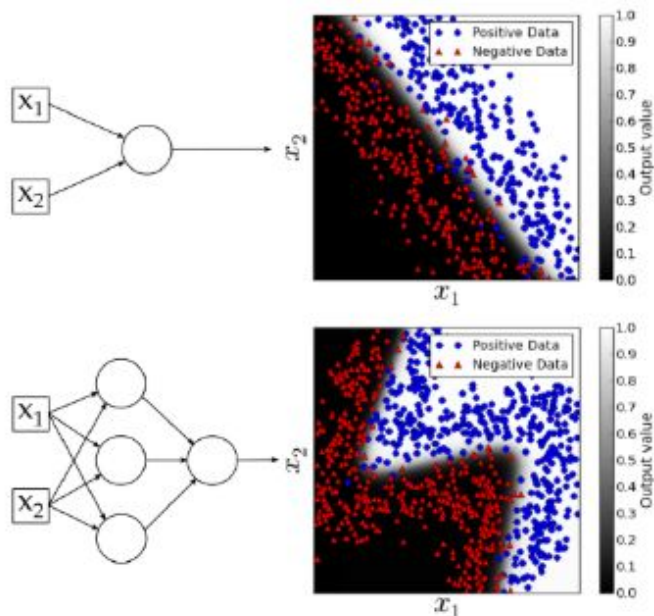
Até que muitos neurônios, podem ajustar decisão até entre grupos e características complexas

	Types of Decision Regions	Exclusive-OR Problem	Classes with Meshed Regions	Most General Region Shapes
Single-Layer 	Half Plane Bounded by Hyperplane			
Two-Layer 	Convex Open or Closed Regions			
Three-Layer 	Arbitrary (Complexity Limited by No. of Nodes)			

Treinamento: Redes Neurais Artificiais

Funcionamento de uma Rede Neural

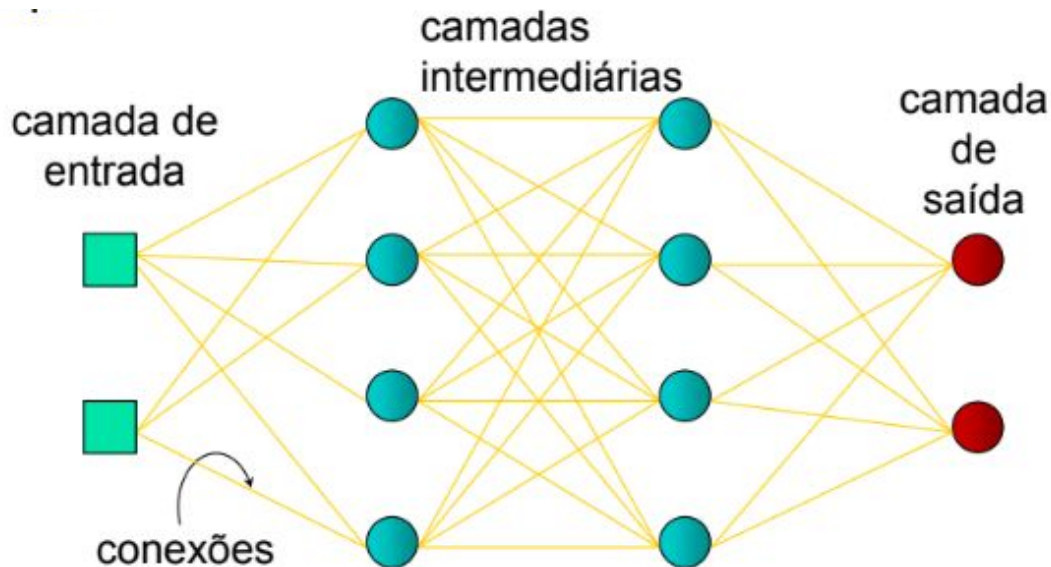
Até que muitos neurônios, podem ajustar decisão até entre grupos e características complexas



Treinamento: Redes Neurais Artificiais

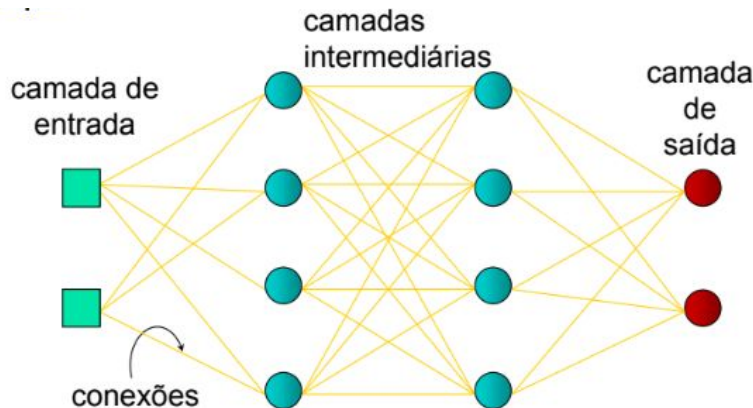
Estrutura geral de uma Rede Neural

A estrutura geral consiste da entrada (as características), as intermediárias onde se realiza o processamento e a saída, onde está as possibilidades de categoria



Treinamento: Redes Neurais Artificiais

Hiperparametros de uma Rede Neural



Número de camadas: Define a profundidade da rede e sua capacidade de abstração de padrões complexos.

Número de neurônios por camada: Influencia o poder de representação e o risco de underfitting ou overfitting

Taxa de aprendizado (η): Controla o quanto os pesos são atualizados a cada iteração do treinamento.

Pesos e viés (bias): São ajustados durante o processo de aprendizado; representam a força das conexões entre neurônios

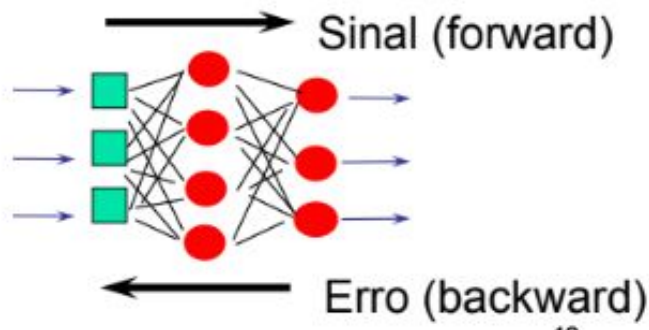
Treinamento: Redes Neurais Artificiais

Treinamento de uma Rede Neural

As Redes Neurais são treinadas com pares entrada-saída

Cada entrada de treinamento é associada à uma saída desejada

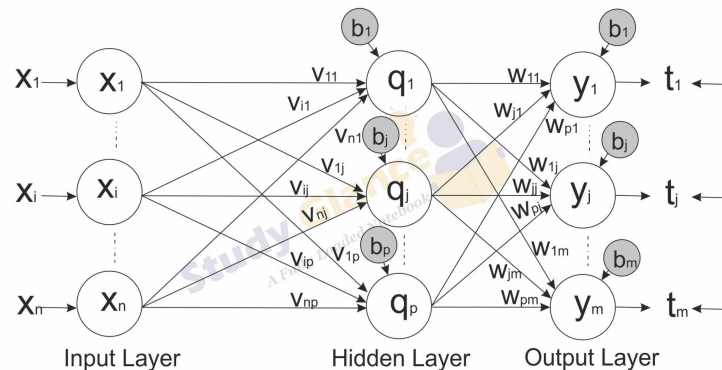
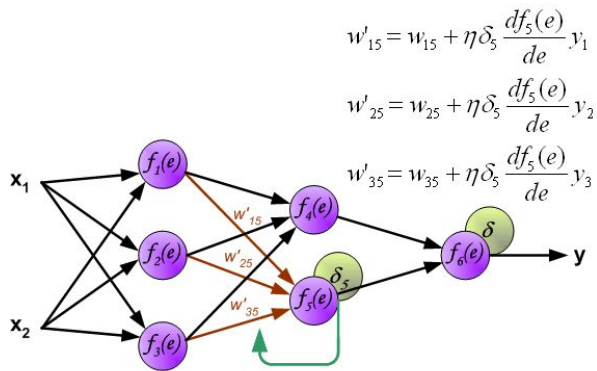
O treinamento é realizado em duas fases, cada uma percorrendo a rede em um sentido: chamado Backpropagation



Treinamento: Redes Neurais Artificiais

Treinamento de uma Rede Neural

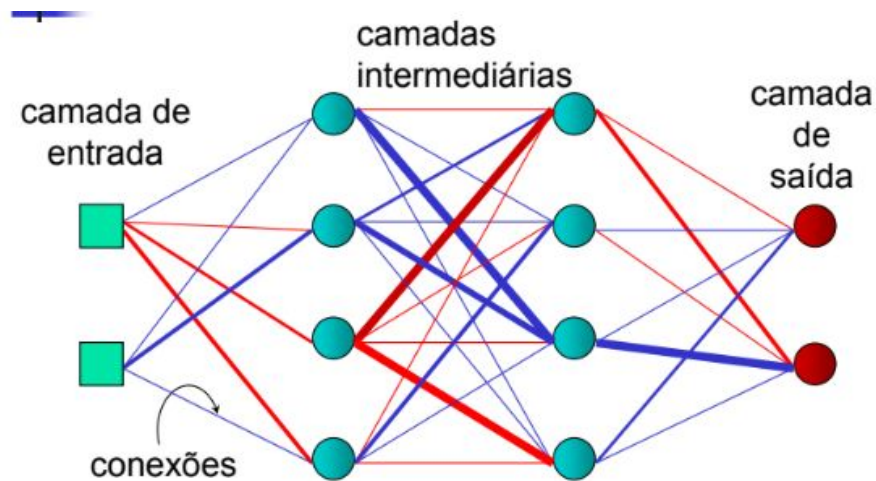
O treinamento é um reajuste de peso, para decisão do melhor caminho



Treinamento: Redes Neurais Artificiais

Treinamento de uma Rede Neural

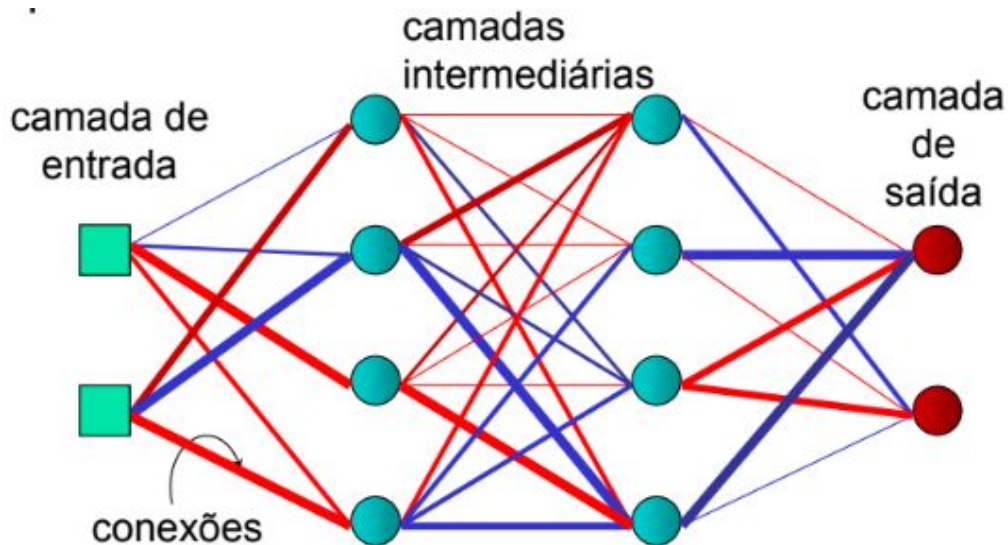
O treinamento é um reajuste de peso, para decisão do melhor caminho



Treinamento: Redes Neurais Artificiais

Treinamento de uma Rede Neural

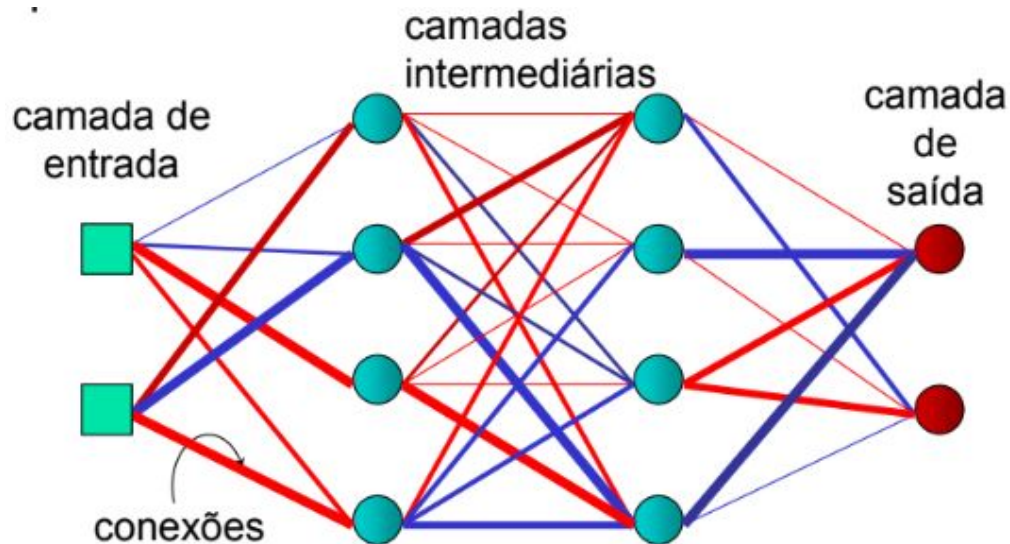
O treinamento é um reajuste de peso, para decisão do melhor caminho
Pesos são atualizados após apresentação de cada exemplo



Treinamento: Redes Neurais Artificiais

Treinamento de uma Rede Neural

Até que em determinado momento os caminhos serão entendidos (os com maiores pesos, alguns caminhos com pesos menores, serão quase irrelevantes em ajudar a separabilidade - veja as linhas mais finas e mais grossas)
Existe até técnicas para podar esses caminhos que não são muito utilizados (Poda Hessiana por exemplo ou Dropout em redes mais modernas) .



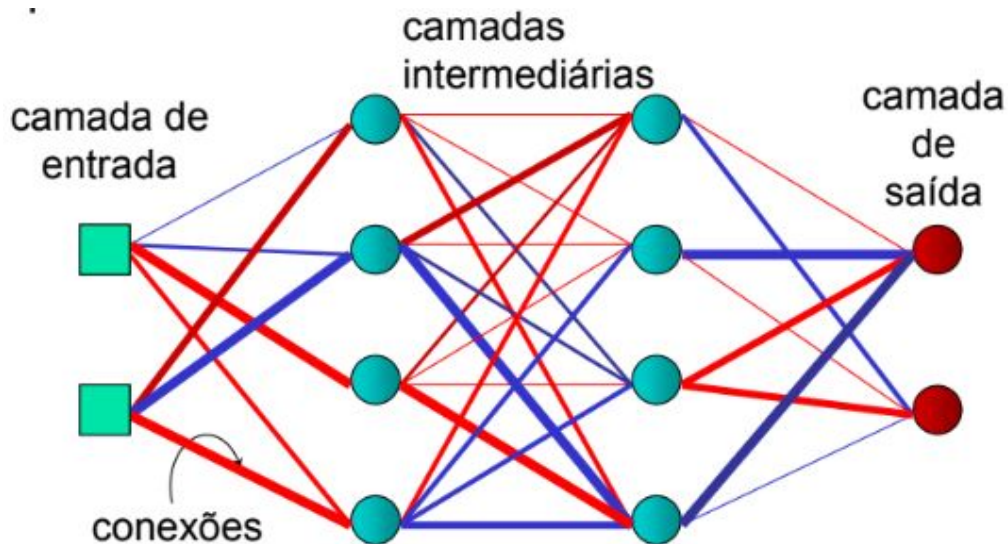
Treinamento: Redes Neurais Artificiais

Treinamento de uma Rede Neural

As conexões mais grossas indicam quais entradas/neuronios influenciam mais a decisão final, não “a decisão em si”.

A decisão é o resultado da soma ponderada de todas as conexões + função de ativação em cada neurônio, camada por camada.

Essas linhas grossas mostram por onde o sinal tem mais força passando até chegar à saída.

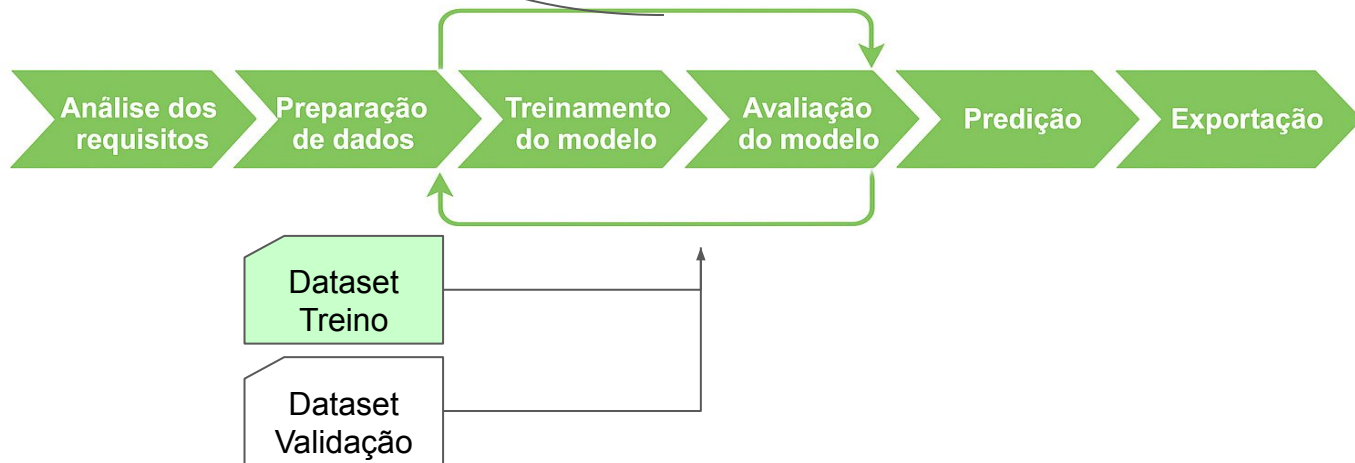


Treinamento: Redes Neurais Artificiais

Treinamento de uma Rede Neural

Um **Ciclo** ou comumente chamada de **Época**, é quando todos os exemplos de **treinamento e validação** passaram pelo modelo.

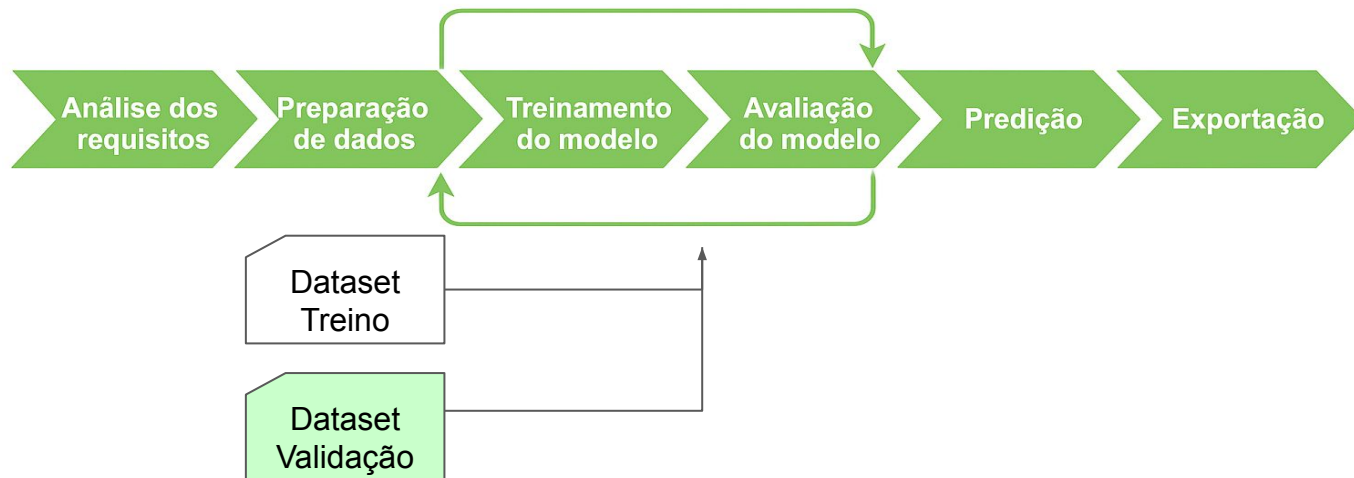
Dados de **treinamento**, são os dados que o modelo vai **estudar**



Treinamento: Redes Neurais Artificiais

Treinamento de uma Rede Neural:

A cada ciclo do treinamento é realizada uma **Validação Cruzada** (com imagens do dataset do treinamento). A validação cruzada mede a saúde em tempo de treinamento. São imagens que não são usadas para treinar, são usadas como um “**simulado**” do seu aprendizado.



Treinamento: Redes Neurais Artificiais

Divisão de um dataset para entrada na Rede Neural

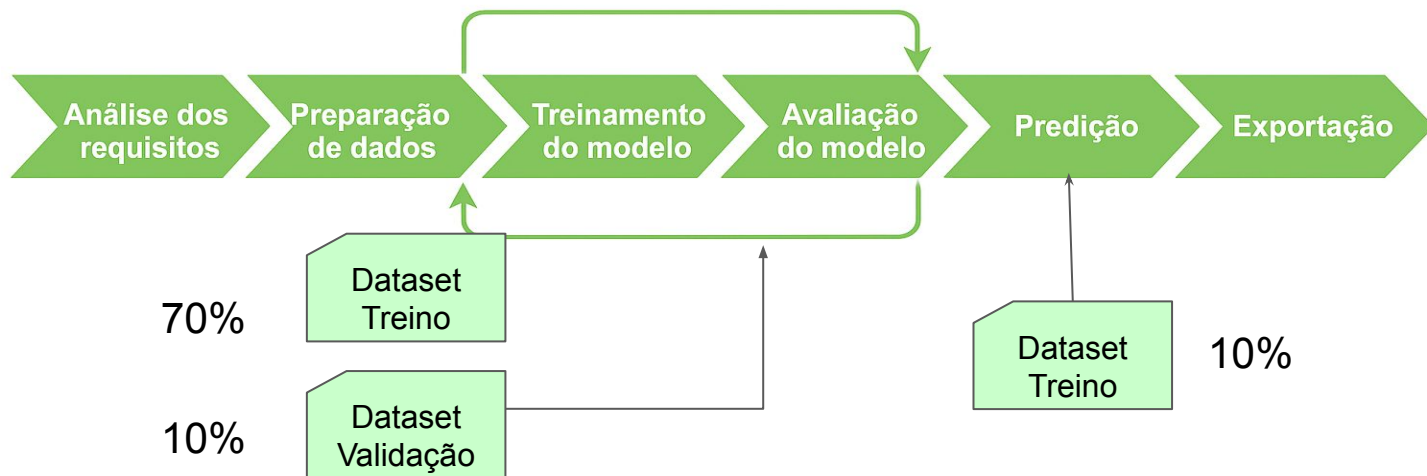
É comum q se prepare um dataset então dividido em Treinamento e Teste
o **Teste é a prova final**, com um dataset que ele nunca viu na fase de treinamento.



Treinamento: Redes Neurais Artificiais

Divisão de um dataset para entrada na Rede Neural

Proporção deste dataset



Rede Neural

Como cada algoritmo "pensa":

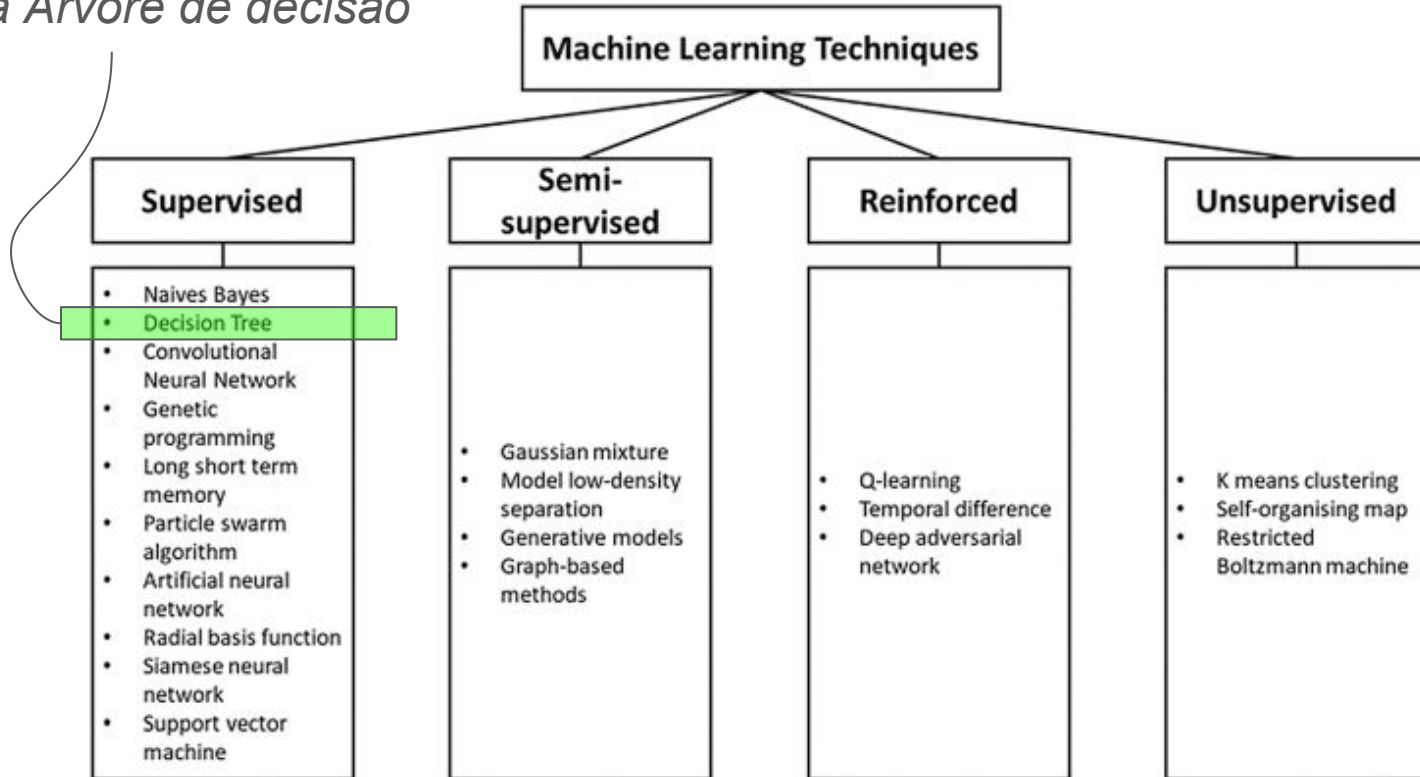
Rede Neural: "Vou traçar vários hiperplanos e combiná-los de forma complexa"

Treinamento: Redes Neurais Artificiais

Atividade: Implementar no Google Colab Notebook uma Rede Neural Artificial (usando um dataset categórico a sua escolha)

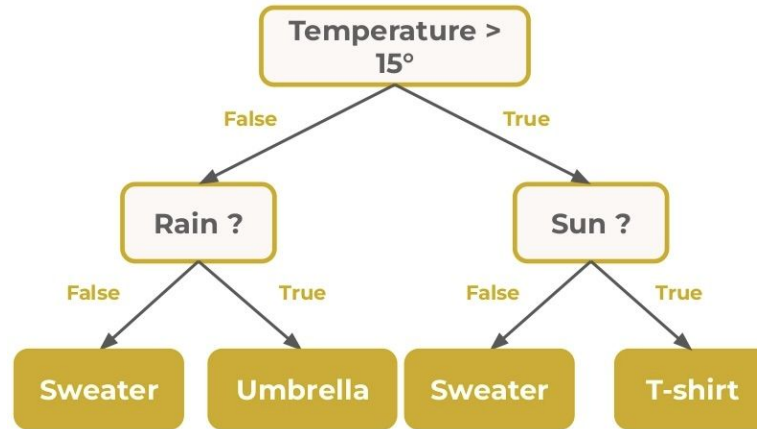
Treinamento: Técnicas para Classificação

*Vamos ver outra técnica,
chamada Árvore de decisão*



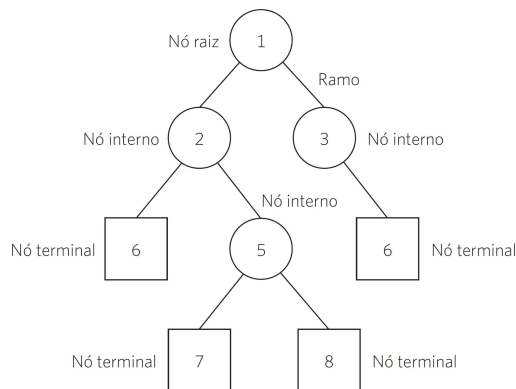
Treinamento: Árvore de Decisão

Árvores de Decisão são algoritmos de aprendizado supervisionado que criam um modelo na forma de árvore para tomar decisões baseadas em características dos dados.



Treinamento: Árvore de Decisão

Componentes de uma Árvore de Decisão



Nó Raiz: Primeira decisão (topo da árvore)

Nós Internos: Decisões intermediárias

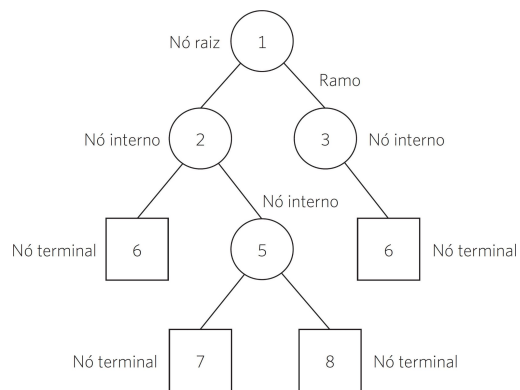
Folhas: **Decisões** finais (classes)

Ramos: Possíveis **resultados** de cada teste

Atributos: **Características** usadas para decidir

Treinamento: Árvore de Decisão

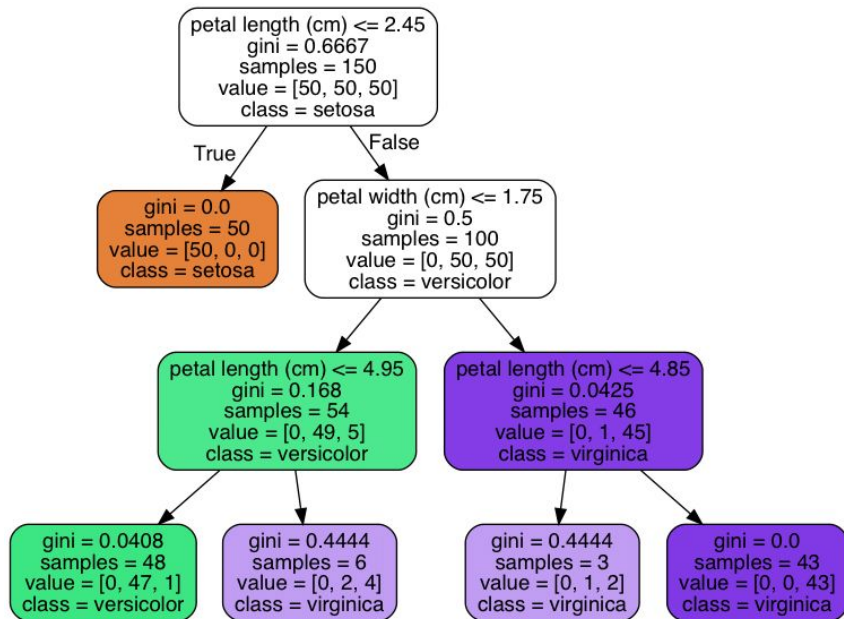
Componentes de uma Árvore de Decisão



- **Nó raiz (Root node):** É o ponto inicial, contendo todos os dados e representando a **primeira pergunta** ou condição a ser verificada
- **Nós internos (Decision nodes):** São os nós intermediários, cada um faz uma pergunta sobre um atributo (por exemplo: "petal length ≤ 2.45 ?") e divide os dados entre os ramos conforme a resposta
- **Ramos (Branches):** São as linhas que conectam os nós e representam os possíveis resultados das perguntas feitas nos nós.
- **Nós folha (Leaf nodes):** São os nós finais e mostram a decisão, classificação, ou resultado. Cada folha representa uma classe (exemplo: "setosa" ou "versicolor") se for um problema de classificação

Treinamento: Árvore de Decisão

Componentes de uma Árvore de Decisão



Petal Length ≤ 2.45 ?

└─ Sim → Setosa

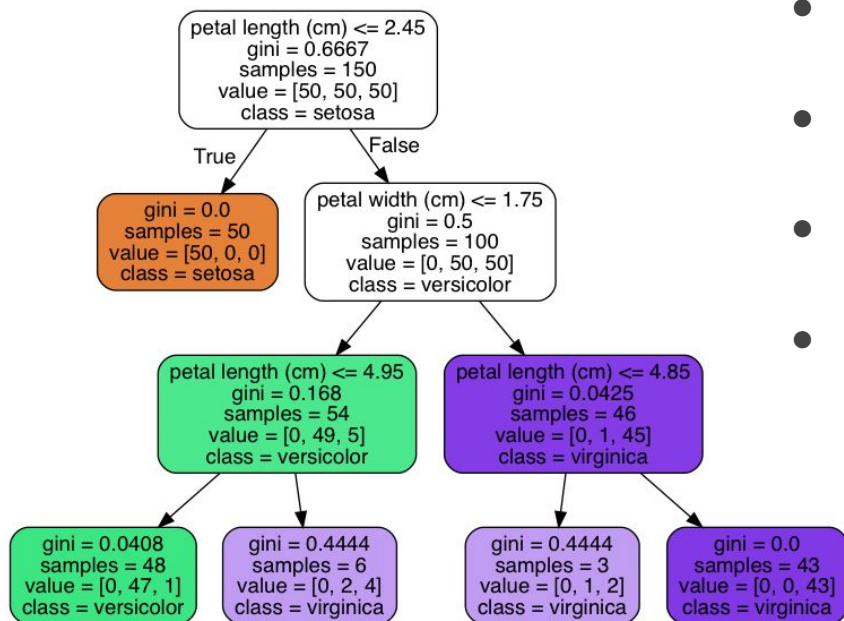
└─ Não → Petal Width ≤ 1.75 ?

└─ Sim → Versicolor

└─ Não → Virginica

Treinamento: Árvore de Decisão

Funcionamento de uma Árvore de Decisão

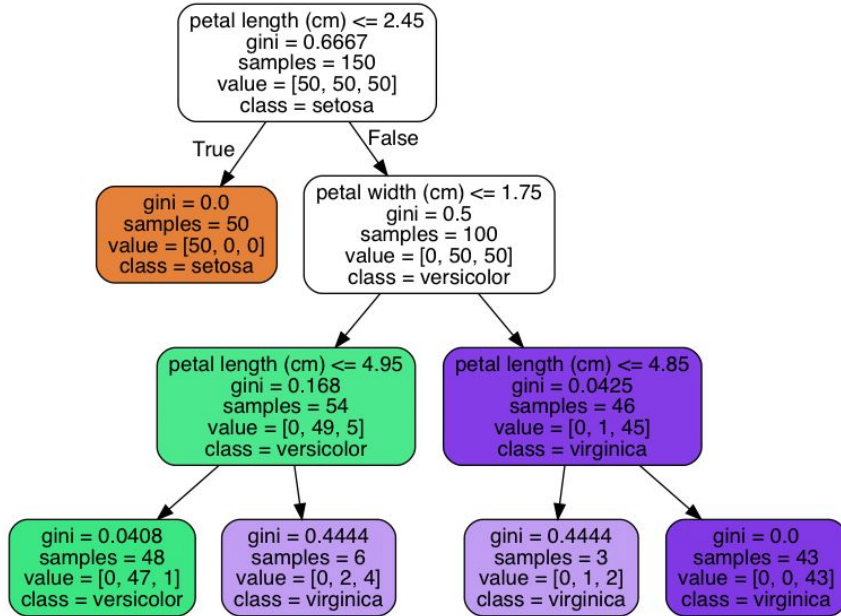


- A árvore começa no nó raiz, verifica uma condição sobre uma feature, e divide os dados.
- Cada subdivisão busca maximizar a pureza dos grupos usando métricas como o gini e a entropia.
- O processo se repete em cada nó interno até atingir uma folha, que traz a resposta do modelo.
- Cada caminho do nó raiz até uma folha representa uma regra de decisão gerada a partir dos dados

Treinamento: Árvore de Decisão

Hiperparâmetros

max_depth: Profundidade máxima da árvore
min_samples_split: Amostras mínimas para dividir um nó
min_samples_leaf: Amostras mínimas em uma folha
max_features: Número máximo de features consideradas
criterion: Critério de divisão (gini, entropy)

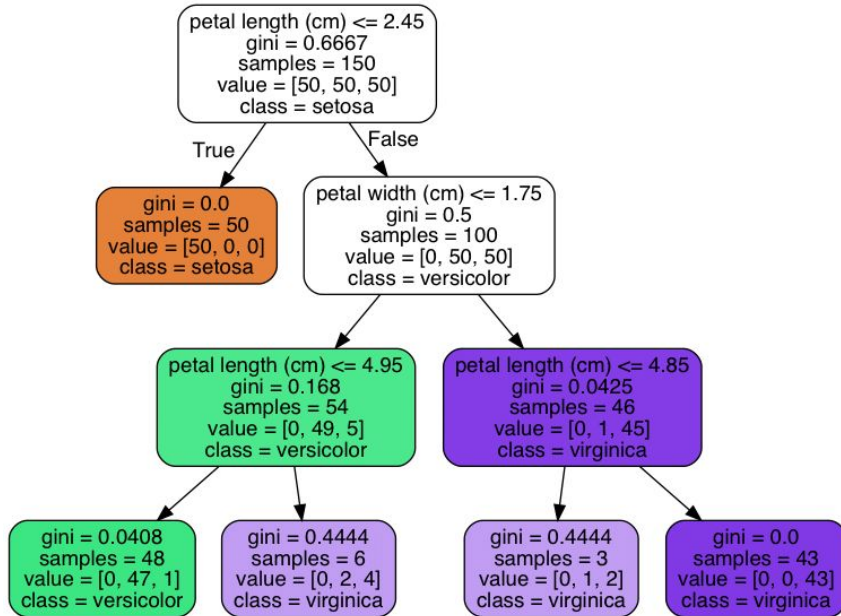


Treinamento: Árvore de Decisão

Hiperparâmetros: Gini

O **gini**: É uma medida de impureza usada para definir onde dividir os nós de uma árvore de decisão

- Valor **proximos ou igual a 0** indica pureza, ou seja, significa que ele tem **predominância (ou exclusividade)** de uma única classe entre seus exemplos.
 - Nó puro (Gini = 0): todas as amostras são da mesma classe, garante que a decisão ali é 100% certa
- Valores mais próximos de 1 indicam maior impureza – significa que naquele nó existem amostras equilibradas de várias classes, então a previsão é incerta e menos informativa
 - Nó impuro (Gini > 0): tem mistura de classes, então as decisões são menos confiáveis.

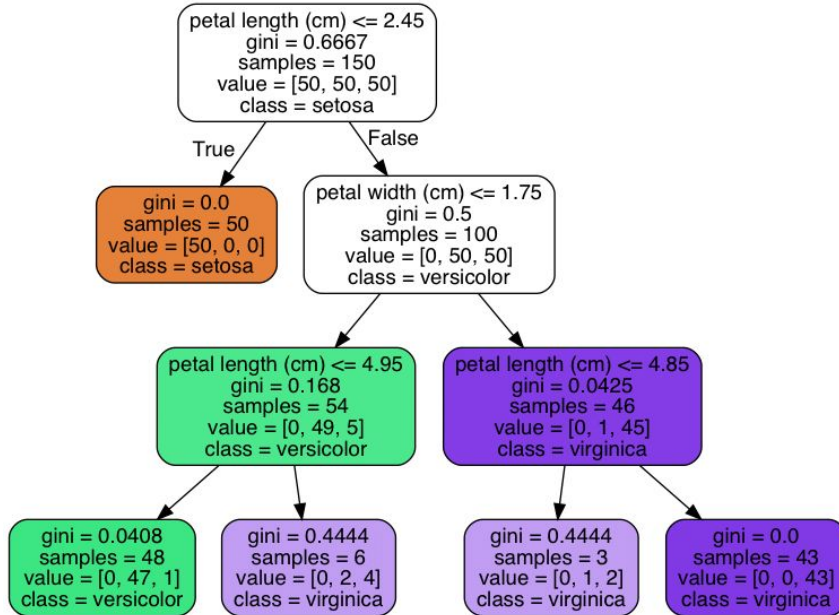


Treinamento: Árvore de Decisão

Hiperparâmetros:

O value:

- O vetor "value" mostra a contagem de amostras de cada classe naquele nó.
- Exemplo: value quer dizer 0 amostras da classe 0 (setosa), 49 da classe 1 (versicolor), e 5 da classe 2 (virginica)
- Isso permite ver qual classe domina cada nó e como os dados foram particionados pela árvore.



Treinamento: Árvore de Decisão

Vantagens das Árvores de Decisão

Interpretável: Fácil de entender e explicar

Não paramétrico: Não assume distribuição dos dados

Lida com dados categóricos e numéricos

Seleção automática de features

Lida bem com dados ausentes

Desvantagem

Tendência ao overfitting

Instável: Pequenas mudanças nos dados podem gerar árvores diferentes

Árvore de Decisão

Como cada algoritmo "pensa":

Rede Neural: "Vou traçar vários hiperplanos e combiná-los de forma complexa"

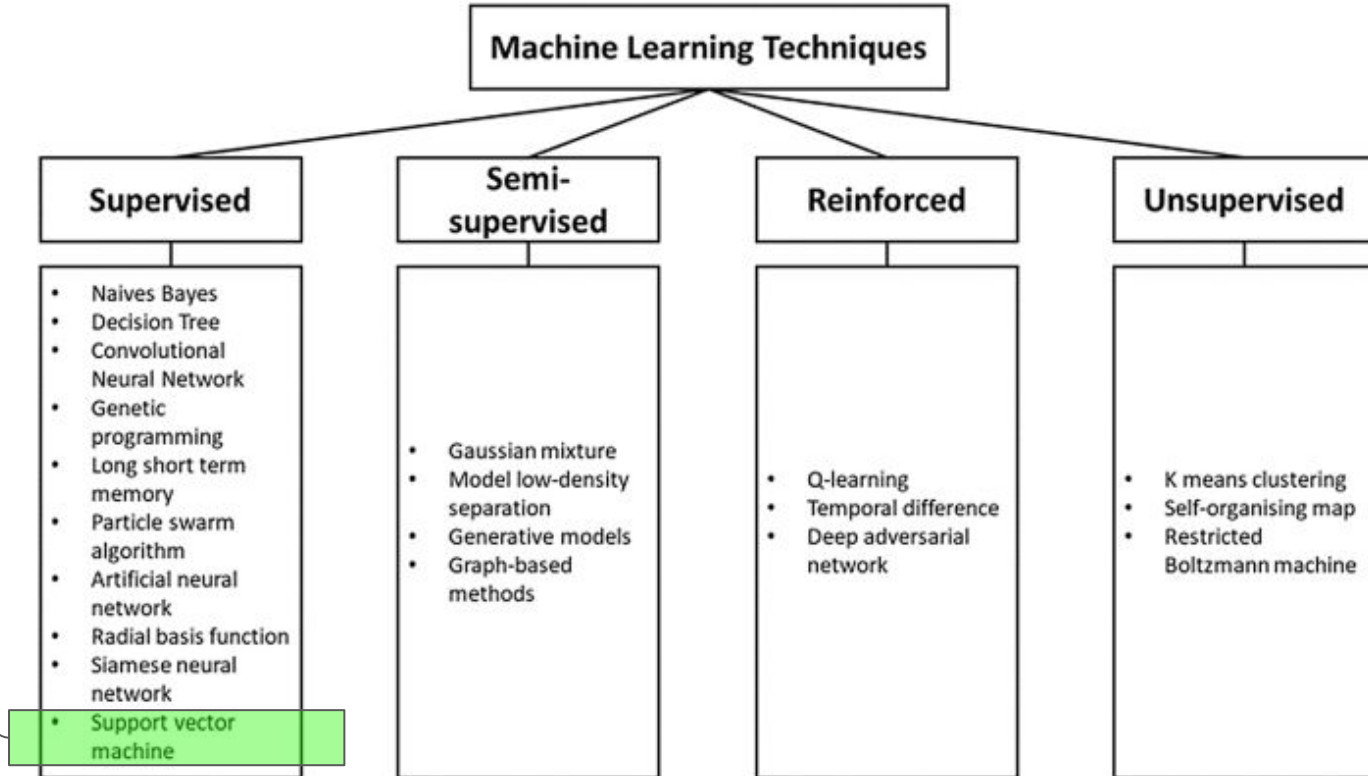
Árvore de Decisão: "Vou fazer uma série de perguntas tipo 'se-então'"

Treinamento: Árvores de Decisão

Atividade: Implementar no Google Colab Notebook uma Árvore de Decisão (usando um dataset categórico a sua escolha)

Treinamento: Técnicas para Classificação

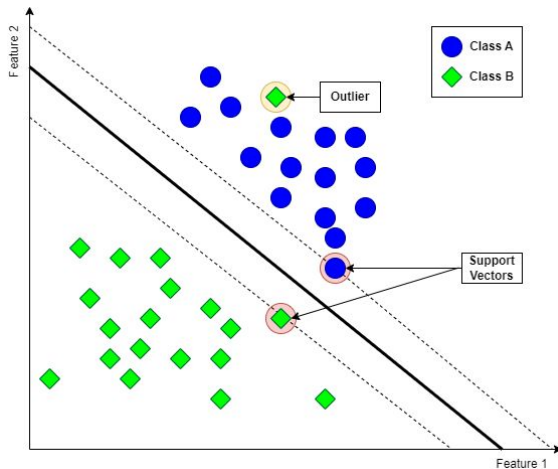
Agora veremos o SVM



Treinamento: Support Vector Machine (SVM)

SVM (Máquina de Vetores de Suporte) é um algoritmo de aprendizado supervisionado que encontra o hiperplano ótimo para separar diferentes classes de dados.

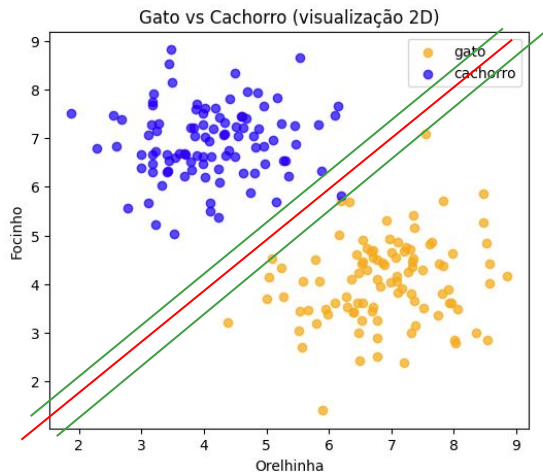
O objetivo é maximizar a margem entre as classes, tornando a classificação mais robusta.



Treinamento: Support Vector Machine (SVM)

Exemplo Visual - Dataset Gato vs Cachorro

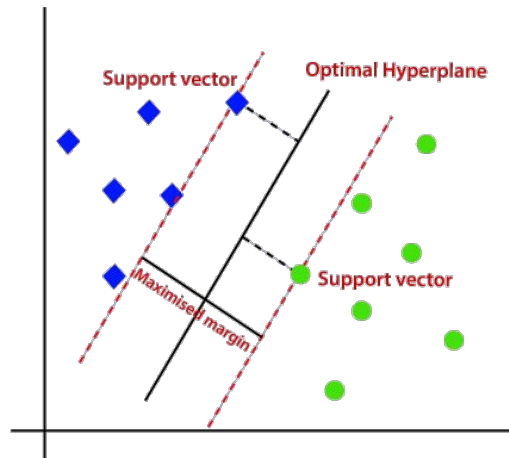
- Assim como nas técnicas anteriores, o SVM também cria fronteiras de decisão no espaço de características (orelhinha vs focinho).



Treinamento: Support Vector Machine (SVM)

Conceitos Fundamentais

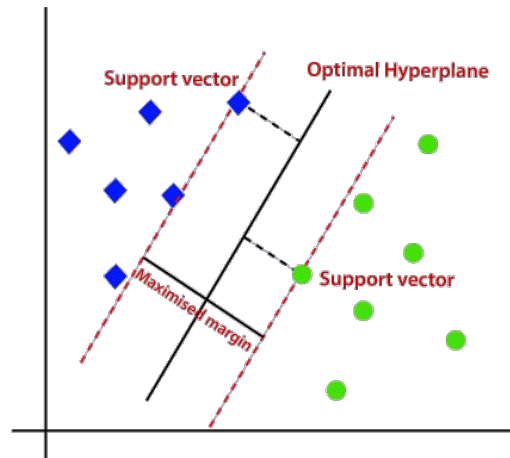
- Hiperplano de Separação: Linha (2D) ou plano (3D+) que divide as classes
- Vetores de Suporte: Pontos mais próximos do hiperplano que definem a margem
- Margem: Distância entre o hiperplano e os vetores de suporte mais próximos
- Margem Máxima: SVM busca o hiperplano com a maior margem possível



Treinamento: Support Vector Machine (SVM)

Funcionamento do SVM

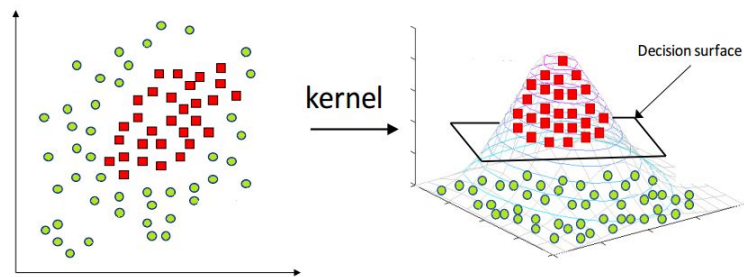
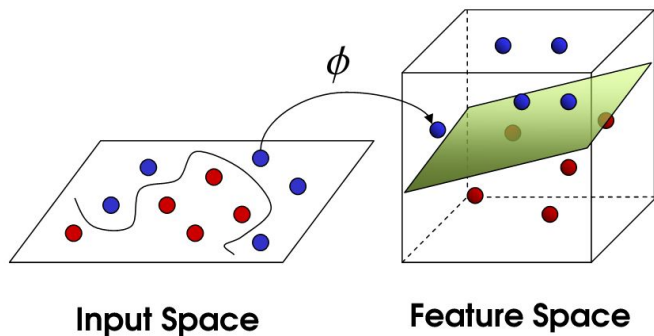
- O SVM funciona encontrando o hiperplano que maximiza a distância (margem) entre as classes.
- Caso Linear: Quando os dados são linearmente separáveis
- Encontra a linha/plano que melhor separa as classes
- Maximiza a distância até os pontos mais próximos de cada classe



Treinamento: Support Vector Machine (SVM)

Funcionamento do SVM - Kernel Trick

- Caso Não-Linear: Quando os dados não são linearmente separáveis
- Usa funções kernel para transformar os dados em dimensões superiores
- No espaço transformado, os dados se tornam linearmente separáveis



Treinamento: Support Vector Machine (SVM)

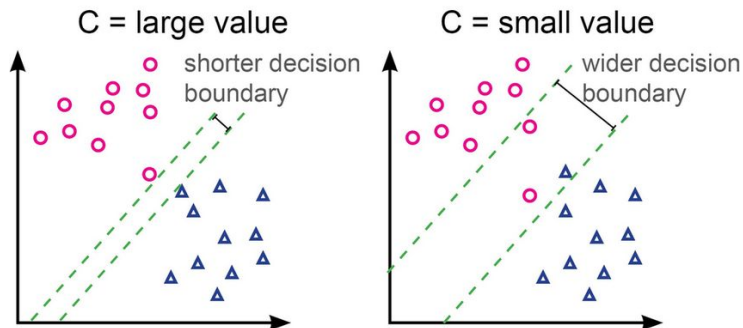
Hiperparâmetros do SVM

- **C** (Parâmetro de Regularização): Controla o trade-off entre maximizar a margem e minimizar erros de classificação
 - C pequeno: Margem maior, tolera mais erros (underfitting)
 - C grande: Margem menor, menos erros no treino (overfitting)
- **kernel**: Tipo de função kernel a ser usada ('linear', 'poly', 'rbf', 'sigmoid')
- **gamma**: Parâmetro para kernels não-lineares (RBF, poly, sigmoid)
 - gamma pequeno: Influência ampla, decisão mais suave
 - gamma grande: Influência local, decisão mais complexa
- **degree**: Grau do polinômio (apenas para kernel polinomial)

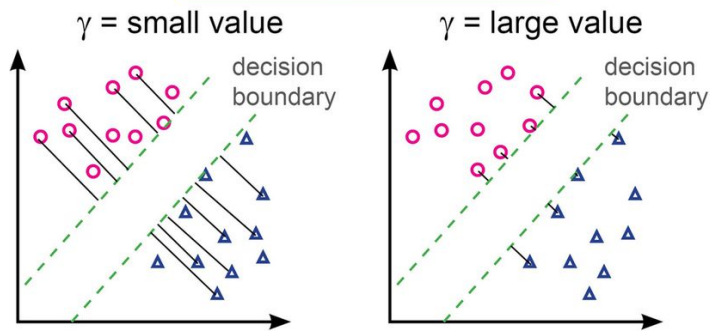
Treinamento: Support Vector Machine (SVM)

Hiperparâmetros do SVM

C parameter



Gamma (γ) parameter



Treinamento: Support Vector Machine (SVM)

Vantagens do SVM

- Eficaz em espaços de alta dimensão
- Usa apenas um subconjunto dos dados (vetores de suporte), sendo eficiente em memória
- Versátil: diferentes funções kernel permitem modelar diversos tipos de fronteiras de decisão
- Robusto a outliers (especialmente com C adequado)

Treinamento: Support Vector Machine (SVM)

Desvantagens

- Computacionalmente custoso em grandes datasets (complexidade $O(n^3)$)
- Sensível à escolha de hiperparâmetros (C, kernel, gamma)
- Não fornece estimativas de probabilidade diretamente
- Difícil de interpretar em casos não-lineares

SVM

Como cada algoritmo "pensa":

Rede Neural: "Vou traçar vários hiperplanos e combiná-los de forma complexa"

Arvore de Decisão: "Vou fazer uma série de perguntas tipo 'se-então' "

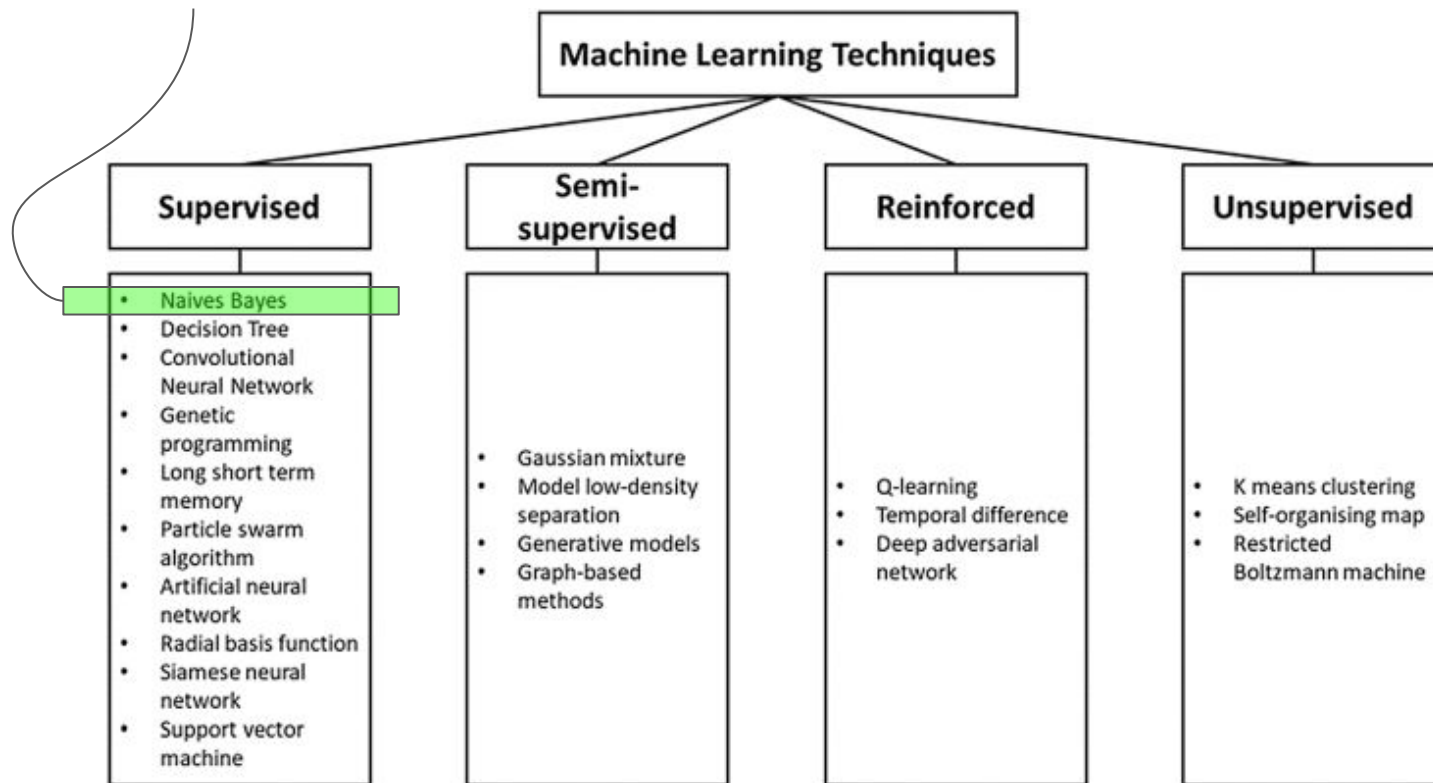
SVM: "Vou encontrar O melhor hiperplano possível com a maior margem"

Treinamento

Atividade: Implementar no Google Colab Notebook um SVM (usando um dataset categórico a sua escolha)

Treinamento: Técnicas para Classificação

Agora veremos o Naive Bayes



Funcionamento: Naive Bayes

Naive Bayes é um algoritmo de classificação baseado no Teorema de Bayes, que calcula a **probabilidade de um dado pertencer a uma determinada classe**.

É chamado de 'Naive' (ingênuo) porque assume que todas as características são independentes entre si, o que simplifica os cálculos.

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Funcionamento: Naive Bayes

LIKELIHOOD

The probability of "B" being True, given "A" is True

PRIOR

The probability "A" being True. This is the knowledge.

$$P(A|B) = \frac{P(B|A).P(A)}{P(B)}$$

POSTERIOR

The probability of "A" being True, given "B" is True

MARGINALIZATION

The probability "B" being True.

- $P(A|B)$: Probabilidade de A dado B (probabilidade posterior)
- $P(B|A)$: Probabilidade de B dado A (verossimilhança)
- $P(A)$: Probabilidade de A (probabilidade a priori)
- $P(B)$: Probabilidade de B (verossimilhança do dado observado)

Funcionamento: Naive Bayes

- $P(A|B)$: Probabilidade de A dado B (probabilidade posterior)
- $P(B|A)$: Probabilidade de B dado A (verossimilhança)
- $P(A)$: Probabilidade de A (probabilidade a priori)
- $P(B)$: Probabilidade de B (verossimilhança do dado observado)

Imagine que queiramos saber a **chance de uma pessoa ter uma doença (A) dado que ela testou positivo em um exame (B)**.

Podemos usar o Teorema de Bayes para incorporar a taxa de pessoas que realmente possuem a doença (probabilidade a priori) e a probabilidade do teste dar positivo SE a pessoa tem a doença (verossimilhança)

Funcionamento: Naive Bayes

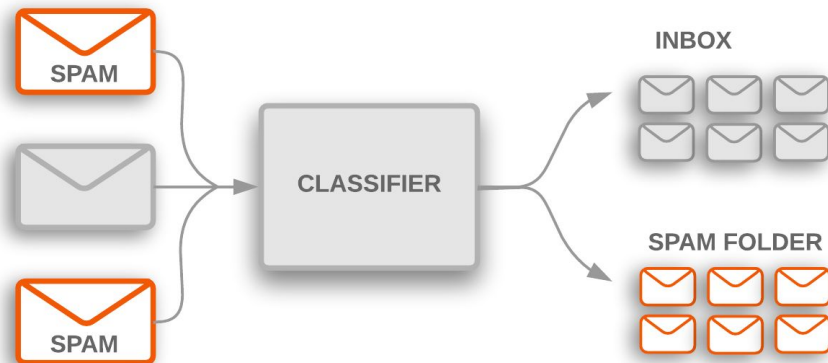
- Para um dado conjunto de dados, ele calcula probabilidades a priori das classes, e as probabilidades condicionalmente independentes das características.
- Para um novo exemplo, estima a probabilidade de pertencer a cada classe multiplicando as probabilidades de cada característica dado a classe, e retorna a classe com maior probabilidade posterior.

Funcionamento: Naive Bayes

Exemplo: Para classificar um email como spam ou não-spam, o algoritmo analisa:

- Probabilidade de ser spam dado que contém a palavra "grátis"
- Probabilidade de ser spam dado que contém números de telefone
- Probabilidade de ser spam dado o remetente

E combina essas probabilidades para fazer a decisão final."



Funcionamento: Naive Bayes

Dataset:

Gato: orelinha=8, focinho=3

Gato: orelinha=7, focinho=4

Cachorro: orelinha=3, focinho=7

Cachorro: orelinha=4, focinho=8

O que o Naive Bayes faz:

1. **APRENDE** a distribuição estatística de cada feature por classe
 - "Gatos geralmente têm orelinha entre 7-9" (calcula média e variância)
 - "Gatos geralmente têm focinho entre 2-4" (calcula média e variância)
 - "Cachorros geralmente têm orelinha entre 2-4"
 - "Cachorros geralmente têm focinho entre 7-9"

Funcionamento: Naive Bayes

Quando chega um novo animal: orelinha=7.5 Ele calcula primeiro:

$$P(\text{Gato} \mid \text{orelinha}=7.5) =$$

$$P(\text{orelinha}=7.5 \mid \text{Gato}) \times$$

$$P(\text{Gato})$$

$$P(\text{Cachorro} \mid \text{orelinha}=7.5) =$$

$$P(\text{orelinha}=7.5 \mid \text{Cachorro}) \times$$

$$P(\text{Cachorro})$$

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Funcionamento: Naive Bayes

Quando chega um novo animal: orelinha=7.5, focinho=3.5 Ele calcula:

$P(\text{Gato} \mid \text{orelinha}=7.5, \text{focinho}=3.5) =$

$P(\text{orelinha}=7.5 \mid \text{Gato}) \times$

$P(\text{focinho}=3.5 \mid \text{Gato}) \times$

$P(\text{Gato})$

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

$P(\text{Cachorro} \mid \text{orelinha}=7.5, \text{focinho}=3.5) =$

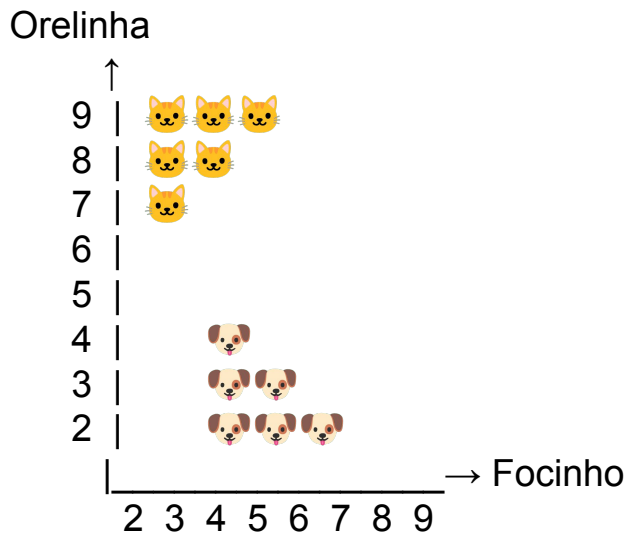
$P(\text{orelinha}=7.5 \mid \text{Cachorro}) \times$

$P(\text{focinho}=3.5 \mid \text{Cachorro}) \times$

$P(\text{Cachorro})$

Treinamento: Naive Bayes

Funcionamento do Naive Bayes



Naive Bayes cria "nuvens de probabilidade":

- Uma nuvem ao redor da região dos gatos
- Uma nuvem ao redor da região dos cachorros

Quando chega um ponto novo, ele vê:

- "Qual a probabilidade desse ponto ter vindo da nuvem dos gatos?"
- "Qual a probabilidade desse ponto ter vindo da nuvem dos cachorros?"

Treinamento: Naive Bayes

Tipos de Naive Bayes

- Gaussian Naive Bayes: Para dados contínuos que seguem distribuição normal (ex: altura, peso, temperatura)
- Multinomial Naive Bayes: Para dados de contagem/frequência (ex: contagem de palavras em textos)
- Bernoulli Naive Bayes: Para dados binários (presença/ausência de características)

Treinamento: Naive Bayes

Hiperparâmetros Naive Bayes

- **var_smoothing** (Gaussian): Controla a suavização da variância para estabilidade numérica
- **alpha** (Multinomial/Bernoulli): Parâmetro de suavização de Laplace
 - $\alpha = 0$: sem suavização
 - $\alpha = 1$: suavização de Laplace (padrão)
 - Valores maiores: mais suavização
- **fit_prior**: Se deve aprender probabilidades a priori das classes"

Treinamento: Naive Bayes

Vantagens

Muito rápido para treinar e fazer previsões

- Funciona bem com datasets pequenos
- Eficiente com alta dimensionalidade (muitas features)
- Não requer muitos dados de treinamento
- Funciona bem com dados categóricos
- Resistente a features irrelevantes
- Fornece probabilidades de previsão"

Treinamento: Naive Bayes

Desvantagens

- A suposição de independência entre features raramente é verdadeira na prática
- Pode ter desempenho ruim quando há forte dependência entre features
- Não é bom para aprender interações complexas entre features
- Sensível ao desbalanceamento de classes
- 'Zero frequency problem': se uma combinação de features nunca apareceu no treino, a probabilidade será zero"

Treinamento: Naive Bayes

Aplicações Comuns

- Classificação de texto e documentos
- Filtro de spam de emails
- Análise de sentimentos
- Sistemas de recomendação
- Categorização de notícias
- Diagnóstico médico (quando features são independentes)

Treinamento: Naive Bayes

Quando usar Naive Bayes?

- ✓ Dataset pequeno ou médio
- ✓ Classificação de texto
- ✓ Features aproximadamente independentes

Treinamento: Naive Bayes

Comparação Naive Bayes vs outras técnicas vista até agora

- vs Redes Neurais: Muito mais rápido, mas menos poderoso
- vs Árvore de Decisão: Mais rápido, mas menos interpretável
- vs SVM: Mais rápido com alta dimensionalidade, mas menos preciso geralmente

Naive Bayes

Como cada algoritmo "pensa":

Rede Neural: "Vou traçar vários hiperplanos e combiná-los de forma complexa"

SVM: "Vou encontrar O melhor hiperplano possível com a maior margem"

Árvore de Decisão: "Vou fazer uma série de perguntas tipo 'se-então'"

Naive Bayes: "Vou calcular probabilidades usando estatística"

Treinamento: Naive Bayes

Atividade 5: Implementar no Google Colab Notebook um Naive Bayes (usando um dataset categórico clássico a sua escolha, Iris, Penguins, etc.)

Treinamento: Técnicas para Classificação

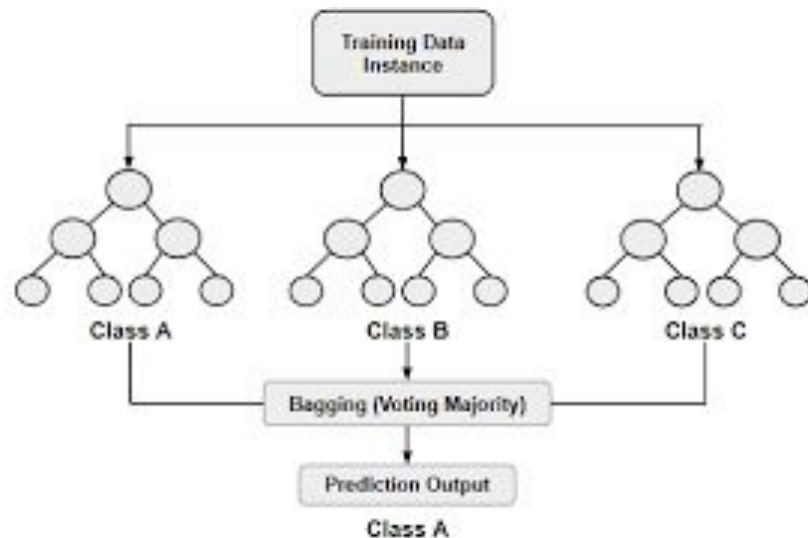
Agora veremos o Random Forest



Outro tipo de aprendizado Supervisionado

O que é: Random Forest

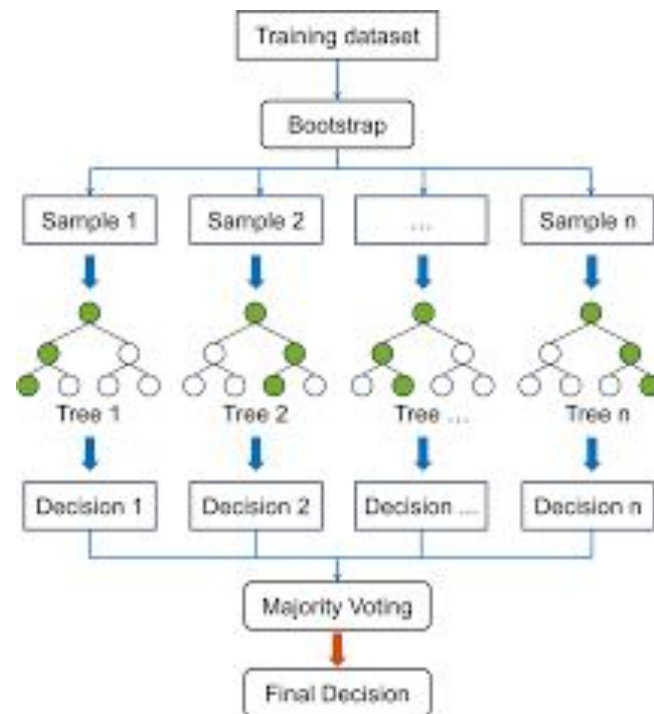
Algoritmo *ensemble* (conjunto) que combina múltiplas árvores de decisão independentes, cada uma treinada em subconjuntos aleatórios dos dados e features. É uma das técnicas de Machine Learning mais poderosas e versáteis para dados tabulares.



O que é: Random Forest

Conceito Principal:

O Random Forest implementa o conceito de "sabedoria das multidões": múltiplos modelos simples (árvores) trabalhando juntos produzem resultados superiores a um único modelo complexo.



Funcionamento: Random Forest

Etapas do Algoritmo:

1. Bootstrap Aggragating:

- Cria N subconjuntos dos dados com reposição (bootstrap)
- Cada árvore treina em um subset diferente
- Garante diversidade nos dados de treino

2. Feature Randomness

- Em cada divisão, considera apenas n características aleatorias
- Impede que características dominantes controlem a árvore

3. Crescimento das Arvores

- Cada arvore cresce até profundidade máxima
- Arvores individuais sao propositalmente especializadas

4. Agregação (Votação)

- Votação majoritária entre as árvores (Bagging)
- Cada árvore vota em uma classe

5. Out-of-Bag Error

- Amostras não usadas no bootstrap (~37%) servem como validação
- Util para avaliar performance durante treino

Hiperparâmetros: Random Forest

Primários

n_estimators:

- Número de árvores na floresta
- Valores típicos: 100-500
- Mais árvores = melhor performance (até certo ponto)
- Trade-off: acurácia vs tempo computacional

max_depth:

- Profundidade máxima de cada árvore
- None = sem limite (árvores crescem completamente)
- Controla complexidade individual das árvores

Hiperparâmetros: Random Forest

Etapas do Algoritmo:

min_samples_split:

- Mínimo de amostras necessárias para dividir um nó
- Valores típicos: 2-10
- Maior valor = árvores mais conservadoras

min_samples_leaf:

- Mínimo de amostras que devem estar em uma folha
- Valores típicos: 1-5
- Previne folhas com muito poucas amostras

max_features:

- Número de features consideradas por divisão
- 'sqrt': raiz quadrada do total (padrão)
- 'log2': logaritmo base 2 do total
- None: todas as features

Hiperparâmetros: Random Forest

Secundários

bootstrap:

- True/False - usar amostragem com reposição
- True é o padrão e recomendado

oob_score:

- Calcular erro out-of-bag durante treino
- Útil para monitorar performance

Vantagens: Random Forest

Principais Vantagens:

Alta Acurácia:

- Consistentemente entre os algoritmos com melhor performance
- Excelente em competições de Machine Learning

Resistente a Overfitting:

- Menos sensível a ruído nos dados

Lida com Não-Linearidade:

- Não assume distribuições específicas dos dados
- Captura relações complexas entre features

Principais Vantagens:

Importância de Features:

- Identifica automaticamente features mais relevantes
- Útil para feature selection e interpretação

Não Precisa Normalização:

- Baseado em divisões, não em distâncias
- Economiza etapa de pré-processamento

Funciona "Out of the Box":

- Hiperparâmetros padrão geralmente funcionam bem
- Poucos ajustes necessários para bons resultados

Lida com Missing Values:

- Pode estimar e trabalhar com valores faltantes
- Robusto a dados incompletos

Desvantagens: Random Forest

Principais Desvantagens:

Menos Interpretável:

- Difícil explicar decisão específica para um caso
- Não gera regras simples de if-then

Mais Lento:

- Treino mais demorado que algoritmos simples
- Predição mais lenta que árvore única
- Cresce linearmente com número de árvores

Memória-Intensivo:

- Modelo pode ocupar bastante espaço (MB)
- Problema em dispositivos com recursos limitados

Tamanho do Modelo:

- Arquivo do modelo pode ser grande
- Dificulta deploy em ambientes restritos

Extrapolação Limitada:

- Não prevê valores fora do range de treino
- Limitado aos padrões vistos nos dados

Pode Overfit:

- Em datasets muito pequenos (< 100 amostras)
- Em dados com muito ruído
- Árvores muito profundas

Aplicações reais: Random Forest

Finanças:

- Detecção de fraude em transações
- Análise de crédito e scoring
- Previsão de inadimplência

Saúde:

- Diagnóstico de doenças
- Identificação de fatores de risco

Marketing:

- Segmentação de clientes
- Propensão de compra

E-commerce:

- Sistemas de recomendação
- Previsão de demanda e vendas
- Precificação dinâmica
- Detecção de fraude

Indústria:

- Manutenção preditiva de equipamentos
- Controle de qualidade
- Otimização de processos
- Detecção de anomalias

Seguros:

- Avaliação de risco
- Precificação de apólices
- Detecção de fraude em sinistros
- Previsão de sinistralidade

Usar e não usar: Random Forest

Usar

Precisa de Alta Acurácia:

- Ambiente de produção
- Decisões críticas de negócio
- Competições de ML

Dataset Médio a Grande:

- Mais de 1000 amostras
- Várias features (dezenas a centenas)
- Dados tabulares estruturados

Features com Relações Complexas:

- Padrões difíceis de capturar

Quer Identificar Features Importantes:

- Feature selection
- Entender quais variáveis importam
- Redução de dimensionalidade

Não usar

Precisa de Interpretabilidade Clara:

- Regulamentação exige explicabilidade
- Auditoria de decisões necessária

Recursos Computacionais Limitados:

- Dispositivos móveis/embarcados
- Memória RAM muito limitada
- Processamento em tempo real crítico
- USE: Naive Bayes ou KNN

Predição em Tempo Real Crítico:

- Latência < 1ms necessária
- Microsegundos importam
- Deploy em sistemas críticos
- USE: Regressão Logística ou Árvore única

Espaço de Armazenamento Crítico:

- Modelo precisa ser muito pequeno
- Deploy em IoT/edge devices
- Bandwidth limitado
- USE: Modelos lineares

Como pensa: Random Forest

Analogia: "Vou criar uma democracia de árvores especialistas: cada árvore dá sua opinião sobre partes diferentes dos dados, e a decisão final é tomada por votação majoritária. A sabedoria da multidão supera especialistas individuais!"

Processo de Decisão:

- Cada árvore analisa o caso sob sua perspectiva única
- Árvores especializadas em diferentes padrões votam
- Votos são agregados democraticamente
- Maioria simples decide a classe final

Por que funciona?

- Erros individuais se cancelam
- Árvores capturam diferentes aspectos dos dados
- Diversidade do ensemble é a chave
- Reduz viés mantendo baixa variância

Como pensa: Random Forest

- **Rede Neural:** *"Vou traçar vários hiperplanos em múltiplas camadas e combiná-los de forma extremamente complexa"*
- **SVM:** *"Vou encontrar O melhor hiperplano possível com a maior margem de separação"*
- **Árvore de Decisão:** *"Vou fazer uma série de perguntas tipo 'se-então' em sequência"*
- **Naive Bayes:** *"Vou calcular probabilidades usando estatística, assumindo independência"*
- **Random Forest:** *"Vou criar uma floresta de árvores especialistas e decidir por votação democrática"*

Como pensa: Random Forest

Algoritmo	Interpretabilidade	Overfitting	Escalabilidade	Funciona Bem Com
Rede Neural	Muito Baixa	Alto	Excelente	Imagens, áudio, texto, big data
SVM	Baixa	Médio	Ruim	Datasets pequenos, alta dimensão
Árvore de Decisão	Muito Alta	Muito Alto	Boa	Dados categóricos, explicação
Naive Bayes	Alta	Baixo	Excelente	Texto, spam, poucos dados
KNN	Média	Baixo	Ruim	Poucos dados, fronteiras complexas
Random Forest	Baixa/Média	Baixo	Boa	Dados tabulares, produção

Como pensa: Random Forest

Algoritmo	Principal Vantagem	Quando É Imbatível
Rede Neural	Aprende representações complexas	Imagens, áudio, texto
SVM	Máxima margem de separação	Dados pequenos + alta dimensão
Árvore de Decisão	100% interpretável	Explicação para stakeholders
Naive Bayes	Extremamente rápido	Baseline, classificação texto
KNN	Zero tempo de treino	Prototipagem rápida
Random Forest	Acurácia + robustez	Dados tabulares em produção

Como pensa: Random Forest

Precisa de velocidade máxima? → Naive Bayes

Precisa explicar cada decisão? → Árvore de Decisão

Precisa de máxima acurácia? → Random Forest

Dataset pequeno e alta dimensão? → SVM

Não quer treinar? → KNN

Trabalhando com imagens/áudio/texto? → Rede Neural

Treinamento: Random Forest

Atividade 7: Implementar no Google Colab Notebook um Random Forest (usando um dataset categórico clássico a sua escolha, Iris, Penguins, etc.)

Treinamento: Técnicas para Classificação

Agora veremos o KNN



Outro tipo de aprendizado Supervisionado

Algoritmo: KNN (K-Nearest Neighbors)

KNN é um algoritmo de aprendizado supervisionado que classifica novos dados com base na similaridade com os dados de treinamento.

O conceito é simples: "Você é semelhante aos seus vizinhos"

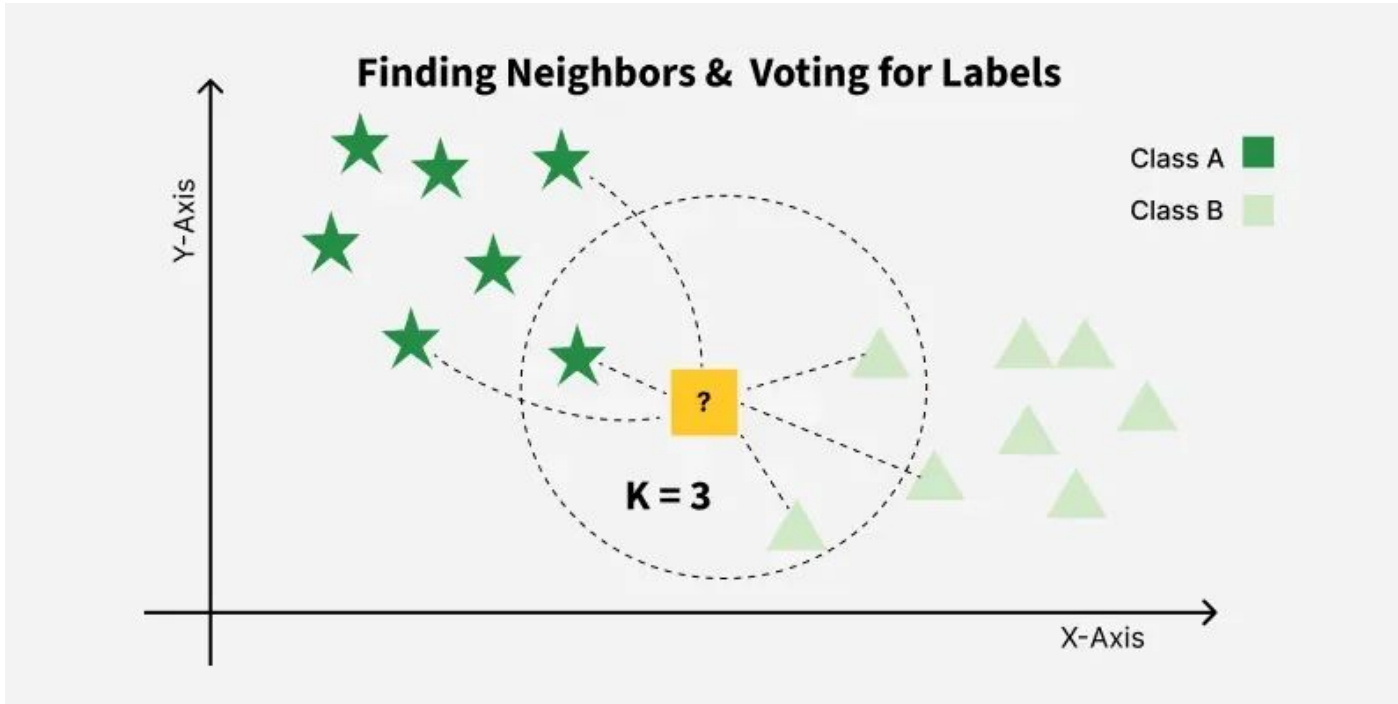
- Não precisa de treinamento explícito (lazy learning)
- Armazena todos os dados de treinamento
- Classifica baseado nos K vizinhos mais próximos

Funcionamento: KNN (K-Nearest Neighbors)

Como funciona:

- Armazena todos os dados de treinamento
- Recebe um novo ponto para classificar
- Calcula a distância entre o novo ponto e todos os pontos do treinamento
- Identifica os K vizinhos mais próximos
- Vota: A classe mais frequente entre os K vizinhos vence

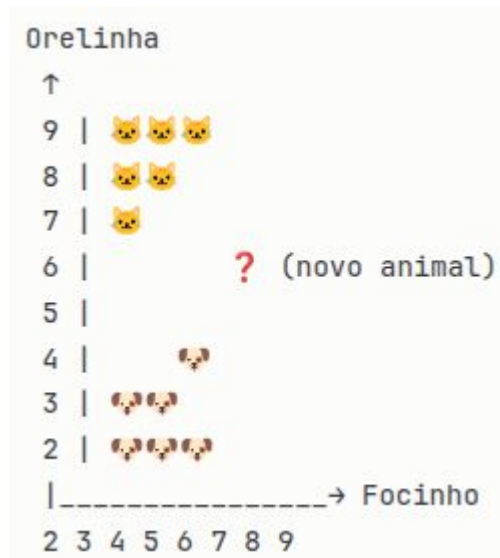
Funcionamento: KNN (K-Nearest Neighbors)



Funcionamento: KNN (K-Nearest Neighbors)

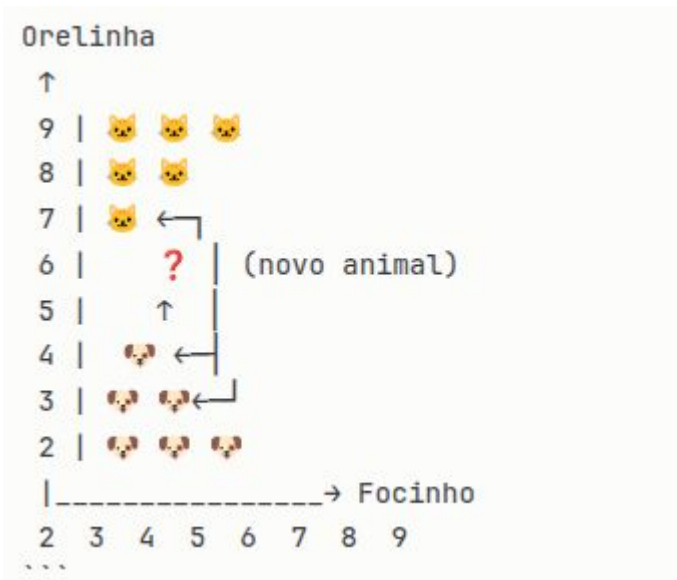
Como o KNN classificaria o ponto ?

Depende de quantos vizinhos (K) considerarmos



Funcionamento: KNN (K-Nearest Neighbors)

Com $K=3$ (considerar os 3 vizinhos mais próximos):



Os 3 vizinhos mais próximos são:

1 🐱 (gato)

2 🐶 (cachorros)

Votação:

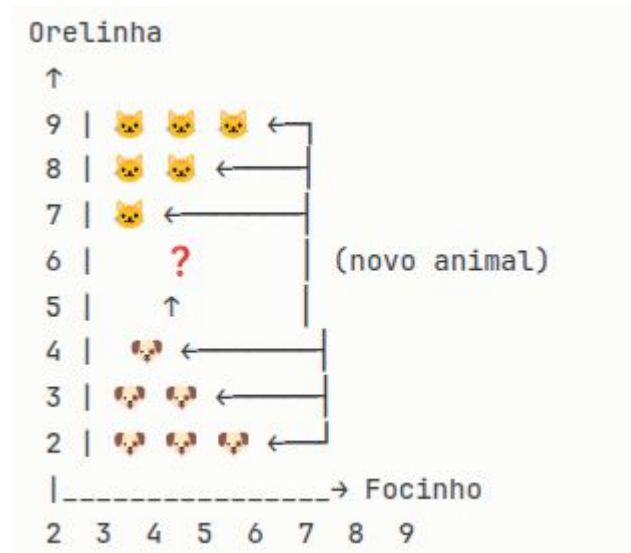
Gatos: 1 voto

Cachorros: 2 votos

Resultado: CACHORRO ✓
(maioria dos votos)

Funcionamento: KNN (K-Nearest Neighbors)

Com $K=7$ (considerar os 7 vizinhos mais próximos):



Os 7 vizinhos mais próximos são:

4 🐱 (gato)

3 🐶 (cachorros)

Votação:

Gatos: 4 votos

Cachorros: 3 votos

Resultado: GATO ✓ (maioria dos votos)

Funcionamento: KNN (K-Nearest Neighbors)

Conclusão:

K menor → decisão local (vizinhos imediatos)

K maior → decisão mais global (região mais ampla)

Funcionamento: KNN (K-Nearest Neighbors)

Como medir "proximidade"?

1. Distância Euclidiana (mais comum - distância em linha reta)

Fórmula: raiz quadrada de $[(x_2-x_1)^2 + (y_2-y_1)^2]$

2. Distância Manhattan (distância em "quarteirões")

Fórmula: $|x_2-x_1| + |y_2-y_1|$

3. Distância Minkowski (generalização das anteriores)

Funcionamento: KNN (K-Nearest Neighbors)

Como medir "proximidade"? – De forma numérica exemplificada

Dataset:

- Gato: orelinha=8, focinho=3
- Cachorro: orelinha=3, focinho=8
- Novo animal: orelinha=7, focinho=4

Calculando distância Euclidiana:

Do novo animal ao Gato:

- $d = \text{raiz de } [(8-7)^2 + (3-4)^2] = \text{raiz de } [1 + 1] = \text{raiz de } 2$
= aproximadamente 1.41

Do novo animal ao Cachorro:

- $d = \text{raiz de } [(3-7)^2 + (8-4)^2] = \text{raiz de } [16 + 16] = \text{raiz de } 32$
= aproximadamente 5.66

Gato está mais próximo! → Classificação: GATO

Funcionamento: KNN (K-Nearest Neighbors)

Comparação visual de K

K=1 (Muito sensível)

- Considera apenas o vizinho mais próximo
- Muito sensível a ruído
- Risco de Overfitting
- Fronteira de decisão irregular

K=3 (Balanceado)

- Considera 3 vizinhos
- Bom equilíbrio
- Fronteira razoável

K=7 (Muito suave)

- Considera 7 vizinhos
- Fronteira muito suave
- Risco de Underfitting
- Perde detalhes importantes

Efeitos do K: KNN (K-Nearest Neighbors)

Visualizando fronteiras de decisão:

K=1: Fronteira muito irregular, captura ruído

K=5: Fronteira razoável, bom equilíbrio

K=15: Fronteira muito suave, perde detalhes

Trade-off:

- K pequeno → Alta variância, baixo viés (overfitting)
- K grande → Baixa variância, alto viés (underfitting)

Precisa encontrar o K ideal através de validação

Hiperparâmetros: KNN (K-Nearest Neighbors)

n_neighbors (k): Quantos vizinhos considerar

- k pequeno (k=1): mais sensível a ruído (overfitting)
- k grande: fronteira mais suave (underfitting)
- k ímpar: evita empates em classificação binária
- Dica: começar com $k = \sqrt{n}$ (n = número de amostras)

metric: Métrica de distância

- 'euclidean' (padrão)
- 'manhattan'
- 'minkowski'

weights: Como ponderar os vizinhos

- 'uniform': todos os vizinhos têm peso igual
- 'distance': vizinhos mais próximos têm mais peso

Normalização: KNN (K-Nearest Neighbors)

Problema: KNN é baseado em distância, então features com escalas diferentes dominam o cálculo.

Exemplo problemático:

- Feature 1: Idade (valores entre 0 e 100)
- Feature 2: Salário (valores entre 0 e 1.000.000)

O Salário domina completamente o cálculo da distância!

Solução: SEMPRE normalizar os dados antes de usar KNN (igual no SVM tinha que normalizar, pois trabalha com o scatter plot das características)

Técnicas de normalização:

- Min-Max Scaling
- Standardization (Z-score)

Vantagens: KNN (K-Nearest Neighbors)

- Simples e intuitivo: fácil de entender e explicar
- Sem fase de treinamento: armazena dados e classifica imediatamente (lazy learning)
- Naturalmente multi-classe: funciona para qualquer número de classes
- Não-paramétrico: não assume distribuição dos dados

Desvantagens: KNN (K-Nearest Neighbors)

- Lento para predição: precisa calcular distância para todos os pontos de treinamento
- Alto uso de memória: armazena todo dataset de treinamento
- PRECISA de normalização: sensível à escala das features
- Curse of dimensionality: desempenho degrada muito com muitas features
- Sensível a desbalanceamento: classes majoritárias dominam
- Sensível a outliers e ruído
- Escolha de K: não é trivial, requer experimentação

Aplicações: KNN (K-Nearest Neighbors)

- Sistemas de recomendação: "usuários similares a você gostaram de..."
- Reconhecimento de padrões: classificação de imagens simples
- Diagnóstico médico: comparar sintomas com casos conhecidos
- Detecção de fraude: transações similares a fraudes conhecidas
- Preenchimento de missing values: usar média dos vizinhos
- Análise de crédito: comparar perfil com outros clientes
- Reconhecimento de escrita: comparar com caracteres conhecidos
- Previsão de preços: imóveis, ações baseado em similares

Quando usar: KNN (K-Nearest Neighbors)

Use KNN quando:

- Dataset pequeno a médio (milhares de exemplos)
- Poucas features (baixa dimensionalidade, menos de 20)
- Fronteiras de decisão complexas e não-lineares
- Precisa incorporar novos dados constantemente
- Interpretabilidade é importante
- Tem tempo para normalizar bem os dados

Evite KNN quando:

- Dataset muito grande (milhões de exemplos)
- Muitas features (alta dimensionalidade)
- Predição precisa ser muito rápida (real-time)
- Features têm escalas muito diferentes e não pode normalizar
- Dados têm muito ruído ou outliers

KNN (K-Nearest Neighbors)

Como cada algoritmo "pensa":

Rede Neural: "Vou traçar vários hiperplanos e combiná-los de forma complexa"

SVM: "Vou encontrar O melhor hiperplano possível com a maior margem"

Árvore de Decisão: "Vou fazer uma série de perguntas tipo 'se-então'"

Naive Bayes: "Vou calcular probabilidades usando estatística"

Random Forest: *"Vou criar uma floresta de árvores especialistas e decidir por votação democrática"*

KNN: "Vou simplesmente perguntar aos meus K vizinhos mais próximos"

Comparação: KNN (K-Nearest Neighbors)

Algoritmo	Como decide	Tempo Treino	Tempo Predição
Rede Neural	Múltiplos hiperplanos combinados	Lento	Rápido
SVM	Hiperplano ótimo (margem máxima)	Lento	Rápido
Árvore de Decisão	Perguntas sequenciais	Rápido	Rápido
Naive Bayes	Probabilidades estatísticas	Rápido	Rápido
KNN	Voto dos K vizinhos	Instantâneo	Lento

KNN é o único "lazy learner" - não aprende durante o treinamento, apenas memoriza!

Treinamento: Redes Neurais Artificiais

Atividade 6: Implementar no Google Colab Notebook um KNN (usando um dataset categórico clássico a sua escolha, Iris, Penguins, etc.)

Agenda 2 da Parte 2

- ❑ Introdução ao Aprendizado Supervisionado
- ❑ Regressão: Conceitos e Fundamentos
 - ❑ Regressão Linear



Etapas 2-3-4:Treinamento/Avaliação/Predição

Tipo de Aprendizado: Aprendizado Supervisionado

Tarefa: Regressão



Etapa 2-3-4 - Treinamento/Avaliação/Predição

Aprendizado Semi supervisionado



Etapa 2-3-4 - Treinamento/Avaliação/Predição

Aprendizado por Reforço

Processo de Desenvolvimento de Modelo de Machine Learning

